Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

MSc Computer Science

Software Security & Engineering - Software Engineering

A serious game for High Performance Computing

Szymon Zinkowicz

Advisor: Prof. Daniele D'Agostino Examiner: Prof. Maura Cerioli

January, 2026

Università di Genova, DIBRIS
Via Opera Pia, 13
16145 Genova, Italy
https://www.dibris.unige.it/

To my father Piotr,
from whom I learned modesty, hard work, and integrity, and whose sacrifices made this possible.

To my mother Wioletta,
who always believed in me and pushed me forward.

To my girlfriend Magdalena,
who has always been patient and caring.

Abstract

High-Performance Computing (HPC) and parallel programming pose significant pedagogical challenges, requiring students to internalize abstractions at the beginning without tangible interactive experiences. Serious games facilitate intuitive understanding through engagement, surfacing misconceptions and enabling targeted instruction.

This thesis presents an educational serious game with the goal to support teachers in presenting fundamental parallel computing concepts—specifically OpenMP (shared memory)—through interactive card-sorting mechanics in web browsers. The game materializes abstract concepts through concrete manipulation: players sort cards collaboratively in multiplayer mode with private buffer zones, simulating OpenMP shared memory with thread-local storage. Students cooperate via a Web interface to minimize sorting time under constraints, learning concepts like speedup, scalability, communication overhead, multithreading, and message passing.

Implemented using the Godot Engine with web export capabilities, this work investigates both pedagogical and technical challenges inherent in the development of web-based educational multiplayer games. The development process involved the identification of a suitable cross-platform game engine, the design of extensible networking mechanisms, and iterative prototyping to balance educational goals with technical feasibility.

Key contributions include: (1) novel pedagogical mapping between card-sorting mechanics and HPC concepts derived from physical classroom experiments, (2) robust multiplayer state synchronization for educational purposes using GDSync, (3) responsive UI/UX solutions for manipulating 50+ interactive elements across screen sizes in web browsers, (4) comprehensive documentation of technical challenges and solutions regarding GDSync and multiplayer synchronization, and (5) an open-source, extensible platform for HPC education research suitable for conference publication and scientific journals.

Keywords: Serious Games, High-Performance Computing, Parallel Computing Education, OpenMP, Web-Based Game Development, Multiplayer Synchronization, Godot Engine, GDScript, Card Sorting, Educational Technology

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Professor Daniele D'Agostino, for his guidance, valuable insights, and continuous support throughout this thesis work. His much needed enthusiasm was what has put me on the rails for High-Performance Computing research. His expertise in High-Performance Computing and educational methodologies has been instrumental in shaping this research.

I am also grateful to Professor Maura Cerioli for serving as the reviewer of this thesis, steering me in right direction regarding organisational aspects and providing constructive feedback.

Special thanks to the Godot Engine community and the developers of the GDSync framework for their open-source contributions and support in resolving technical issues encountered during development.

Finally, I thank my family and friends for their encouragement and patience during the completion of this work.

Contents

Abstract **5**

Acknowledgements **7**

Chapter 1 Introduction **19**

1.1 Context and Motivation . . . 19
1.1.1 The Challenge of Teaching High-Performance Computing . . . 19
1.1.2 Serious Games as Educational Tools . . . 20
1.1.3 From Physical Experiments to Digital Games . . . 20
1.2 Problem Statement . . . 21
1.3 Research Objectives . . . 22
1.3.1 Core Objectives . . . 22
1.4 Proposed Solution: HPC Sorting Serious Game . . . 22
1.4.1 Core Concept . . . 23
1.4.2 Technology Stack . . . 23
1.5 Thesis Contributions . . . 24

1.6 Thesis Organization 24

Chapter 2 Background and Literature Review 27
2.1 Parallel Computing Fundamentals 27
2.1.1 Parallel Programming Paradigms 27
2.1.2 OpenMP: Shared-Memory Programming 28
2.1.3 MPI: Distributed-Memory Programming 29
2.1.4 Parallel Sorting Algorithms 30
2.2 Serious Games in Education 31
2.2.1 Definition and Characteristics 31
2.2.2 Theoretical Foundations 31
2.2.3 Empirical Evidence 32
2.2.4 Serious Games in Computer Science 33
2.2.5 Serious Games for HPC Education 33
2.3 Web-Based Game Development 35
2.3.1 Web Platform Advantages 35
2.3.2 Web Development Challenges 35
2.3.3 Game Engines for Web 35
2.4 Multiplayer Game Architecture 36
2.4.1 Client-Server Architecture 36
2.4.2 Peer-to-Peer Architecture 37
2.4.3 Web Transport Technologies 37
2.4.4 State Synchronization Patterns 38
2.5 Related Work 39
2.5.1 Parallel Computing Educational Tools 39
2.5.2 Serious Games for Computing Education 40
2.5.3 Physical Teaching Methods 41
2.6 Summary 42

Chapter 3 Methodology 45
3.1 Research Methodology 45
3.1.1 Design Science Research Framework 45
3.1.2 Development Approach 46
3.2 Requirements Analysis 47
3.2.1 Educational Requirements 47
3.2.2 Functional Requirements 48
3.3 Technology Selection 50
3.3.1 Game Engine: Godot 4.x 50
3.3.2 Programming Language: GDScript 52
3.3.3 Multiplayer Framework: GDSync 53
3.3.4 Networking Technology: HTTP/SSE Relay 54
3.3.5 Supporting Plugins and Tools 55
3.4 Game Design Methodology 57
3.4.1 Mapping HPC Concepts to Game Mechanics 57
3.4.2 User Experience Design 57
3.5 Evaluation Methodology 58
3.6 Development Tools and Environment 59

3.7 Summary 60

Chapter 4 System Design and Architecture **61**
4.1 Architectural Overview 61
4.1.1 Design Principles 61
4.1.2 System Layers 62
4.2 Scene Structure 62
4.2.1 Main Menu Scene 62
4.2.2 Single-Player Game Scene 63
4.2.3 Multiplayer Lobby Scene 64
4.2.4 Multiplayer Game Scene 66
4.3 Core Components 68
4.3.1 Card Component 68
4.3.2 Card Manager 69
4.3.3 Multiplayer Card Manager 72
4.3.4 Scroll Container and Layout 75
4.4 Multiplayer Architecture 76
4.4.1 Network Topology 76
4.4.2 State Synchronization Patterns 76
4.4.3 GDSync Integration 77
4.4.4 Connection Management 78
4.5 Data Flow Diagrams 80
4.5.1 Single-Player Game Flow 80
4.5.2 Multiplayer Card Movement Flow 80
4.5.3 Buffer Visibility Synchronization 82
4.6 Responsive UI/UX Design 83
4.6.1 Design Constraints 83
4.6.2 Layout Strategies 84
4.6.3 Touch Interaction Patterns 84
4.6.4 Visual Feedback Systems 85
4.7 Performance Considerations 85
4.7.1 Optimization Strategies 85
4.7.2 Scalability Limits 86
4.8 Summary 86

Chapter 5 Implementation **89**
5.1 Development Environment Setup 89
5.1.1 Project Structure 89
5.2 Core Components Implementation 90
5.2.1 Card Component 90
5.2.2 Card Manager 90
5.2.3 Buffer System 91
5.3 Single-Player Mode Implementation 91
5.3.1 Game Initialization 91
5.3.2 Drag-and-Drop System 91
5.3.3 Sorting Validation 92
5.4 Multiplayer Mode Implementation 92

5.4.1 Lobby System . . . 92
5.4.2 GDSync Integration . . . 92
5.4.3 State Synchronization . . . 93
5.4.4 Visibility Management . . . 94
5.5 Mobile Optimizations . . . 94
5.5.1 Performance Optimizations . . . 94
5.5.2 Touch Input Optimization . . . 95
5.5.3 Network Optimization . . . 95
5.6 Debugging and Development Tools . . . 95
5.6.1 Logger Plugin Integration . . . 95
5.6.2 Runtime Variable Inspection . . . 95
5.7 Code Quality and Best Practices . . . 96
5.7.1 Code Organization . . . 96
5.7.2 Documentation . . . 96
5.8 Summary . . . 96

Chapter 6 Problems and Challenges 99
6.1 Overview . . . 99
6.2 Technology Selection Challenges . . . 99
6.2.1 Game Engine Trade-offs . . . 99
6.2.2 GDScript Performance Concerns . . . 100
6.3 GDSync Framework Challenges . . . 101
6.3.1 Protected Mode Blocking Issue . . . 101
6.3.2 Incomplete Documentation . . . 102
6.3.3 Debugging Multiplayer Issues . . . 103
6.4 Multiplayer Synchronization Challenges . . . 103
6.4.1 Card Order Synchronization . . . 103
6.4.2 Timing and Race Conditions . . . 104
6.4.3 Different Client Views . . . 105
6.5 Mobile UI/UX Challenges . . . 105
6.5.1 Displaying Many Cards on Small Screens . . . 105
6.5.2 Touch Interaction Precision . . . 106
6.5.3 Performance on Low-End Devices . . . 107
6.6 Testing Challenges . . . 107
6.6.1 Multiplayer Testing Complexity . . . 107
6.6.2 Lack of Automated Testing . . . 108
6.7 Educational Design Challenges . . . 109
6.7.1 Balancing Accuracy and Playability . . . 109
6.7.2 Assessment of Learning Effectiveness . . . 109
6.8 Lessons Learned . . . 110
6.8.1 Technical Lessons . . . 110
6.8.2 Design Lessons . . . 110
6.8.3 Process Lessons . . . 111
6.9 Summary . . . 111

Chapter 7 Results and Evaluation 113
7.1 Completed System Features . . . 113

7.1.1 Functional Features . . . 113
7.1.2 Educational Features . . . 114
7.2 Technical Performance . . . 114
7.2.1 Frame Rate Performance . . . 114
7.2.2 Memory Usage . . . 115
7.2.3 Web Export Size . . . 116
7.3 Platform Compatibility . . . 116
7.3.1 Browser Compatibility . . . 116
7.3.2 Responsive Layout . . . 117
7.4 Usability Evaluation . . . 117
7.4.1 Informal User Testing . . . 117
7.4.2 Key Findings . . . 118
7.4.3 Suggested Improvements . . . 119
7.5 Educational Effectiveness . . . 119
7.5.1 Preliminary Assessment . . . 119
7.5.2 Pedagogical Value Proposition . . . 121
7.5.3 Comparison with Traditional Methods . . . 121
7.6 Comparison with Initial Requirements . . . 121
7.6.1 Requirements Fulfillment . . . 121
7.7 Known Limitations . . . 122
7.7.1 Current Limitations . . . 122
7.7.2 Scope Limitations . . . 123
7.8 Summary . . . 123

Chapter 8 Conclusion and Future Work 125
8.1 Summary of Contributions . . . 125
8.1.1 Primary Contributions . . . 125
8.2 Research Questions Revisited . . . 126
8.2.1 Addressing the Educational Problem . . . 126
8.2.2 Addressing the Technical Problem . . . 127
8.2.3 Addressing the Design Problem . . . 127
8.3 Implications . . . 128
8.3.1 For HPC Education . . . 128
8.3.2 For Serious Game Development . . . 129
8.4 Future Work . . . 129
8.4.1 Short-Term Enhancements . . . 129
8.4.2 Medium-Term Extensions . . . 130
8.4.3 Long-Term Research Directions . . . 132
8.4.4 Community and Ecosystem Development . . . 133
8.5 Limitations and Reflections . . . 134
8.5.1 Methodological Limitations . . . 134
8.5.2 Reflections on the Development Process . . . 135
8.6 Concluding Remarks . . . 135
8.7 Final Thoughts . . . 136

Bibliography 137

Appendix A User Manual **139**
A.1 Introduction . . . 139
A.2 System Requirements . . . 139
A.2.1 Minimum Requirements . . . 139
A.2.2 Recommended Requirements . . . 139
A.3 Installation . . . 140
A.3.1 Installing from APK . . . 140
A.3.2 Installing from Google Play (Future) . . . 140
A.4 Getting Started . . . 140
A.4.1 First Launch . . . 140
A.5 Single-Player Mode . . . 141
A.5.1 Starting a Single-Player Game . . . 141
A.5.2 Gameplay . . . 141
A.6 Multiplayer Mode . . . 142
A.6.1 Creating a Game . . . 142
A.6.2 Joining a Game . . . 142
A.6.3 Multiplayer Gameplay . . . 142
A.7 Settings . . . 143
A.7.1 Game Settings . . . 143
A.7.2 Audio Settings . . . 143
A.7.3 Display Settings . . . 143
A.7.4 Network Settings . . . 144
A.8 Tips and Strategies . . . 144
A.8.1 Efficient Sorting Strategies . . . 144
A.8.2 Multiplayer Coordination . . . 144
A.9 Troubleshooting . . . 144
A.9.1 Common Issues and Solutions . . . 144
A.10 Educational Use . . . 145
A.10.1 For Students . . . 145
A.10.2 For Instructors . . . 146
A.11 Support and Resources . . . 146
A.11.1 Getting Help . . . 146
A.11.2 Contributing . . . 146

Appendix B Code Listings **147**
B.1 Source Code Repository . . . 147
B.2 Project Structure . . . 147
B.3 Key Components . . . 148
B.3.1 Card System . . . 148
B.3.2 Networking . . . 148
B.3.3 Game Logic . . . 148
B.4 Configuration . . . 148
B.4.1 Web Export Settings . . . 148
B.4.2 GDSync Configuration . . . 149
B.5 Building and Running . . . 149
B.5.1 Development . . . 149
B.5.2 Web Export . . . 149

B.5.3 Relay Server . . . 149

Appendix C API Documentation **151**
C.1 Introduction . . . 151
C.2 Core Game Classes . . . 151
C.2.1 Card Class . . . 151
C.2.2 CardManager Class . . . 152
C.2.3 Buffer Class . . . 152
C.3 Multiplayer Classes . . . 153
C.3.1 LobbyManager Class . . . 153
C.3.2 NetworkSync Class . . . 154
C.4 GDSync Framework API . . . 155
C.4.1 Sync Annotations . . . 155
C.4.2 RPC Annotations . . . 155
C.4.3 Connection Management . . . 156
C.5 Plugin APIs . . . 156
C.5.1 ToastParty API . . . 156
C.5.2 Logger API . . . 157
C.6 Utility APIs . . . 157
C.6.1 Profiler API . . . 157
C.6.2 Config Manager API . . . 158
C.7 Event System . . . 158
C.7.1 Global Events . . . 158
C.8 Extension Points . . . 159
C.8.1 Custom Game Modes . . . 159
C.8.2 Custom Card Types . . . 159
C.9 Development Tools . . . 159
C.9.1 Debug Commands . . . 159

Appendix D Study Materials **161**
D.1 Introduction . . . 161
D.2 Learning Objectives . . . 161
D.2.1 Foundational Concepts . . . 161
D.2.2 Sequential Execution Concepts (Single-Player Mode) . . . 161
D.2.3 OpenMP Concepts (Multiplayer Mode) . . . 162
D.3 Discussion Questions . . . 162
D.3.1 Pre-Game Questions . . . 162
D.3.2 Post-Game Discussion: Single-Player Mode . . . 163
D.3.3 Post-Game Discussion: Multiplayer Mode . . . 163
D.3.4 Connecting to HPC Concepts . . . 163
D.4 Classroom Activities . . . 164
D.4.1 Activity 1: Strategy Competition . . . 164
D.4.2 Activity 2: Does More Players Mean Faster? . . . 164
D.4.3 Activity 3: Working Together vs. Working Separately . . . 165
D.5 Exercises . . . 165
D.5.1 Exercise 1: Comparing Single and Multiple Players . . . 166
D.5.2 Exercise 2: Finding the Best Team Size . . . 166

D.5.3 Exercise 3: Dividing the Work 167
D.6 Conceptual Mappings 167
D.6.1 Game to OpenMP 167
D.7 Assessment Rubric 167
D.7.1 Conceptual Understanding (40 points) 168
D.7.2 Practical Application (30 points) 168
D.7.3 Analysis and Reflection (30 points) 168
D.8 Additional Resources 168
D.8.1 Recommended Readings 168
D.8.2 Online Resources 168
D.8.3 Hands-On Practice 169
D.9 Instructor Notes 169
D.9.1 Integration with Curriculum 169
D.9.2 Common Student Misconceptions 170

List of Figures

1.1 Transformation from physical classroom experiments to digital serious game, preserving pedagogical effectiveness while overcoming physical limitations 21
1.2 Comparison of game modes: Single-player represents sequential execution (left) where one player sorts all cards independently; Multi-player represents OpenMP shared-memory parallelism (right) where multiple players collaborate with a shared container and private buffers 23

3.1 HTTP/SSE relay message flow: connection, lobby creation, and gameplay synchronization 56

4.1 Main menu interface showing the game title, mode selection buttons, and animated card decorations 63
4.2 Single-player game interface showing numbered cards in the shared container and empty buffer zones below 65
4.3 Single-player game with a card (100) placed in the buffer zone, demonstrating thread-local storage simulation 65
4.4 HTTP/SSE relay architecture: star topology with centralized relay server 78

5.1 VarTree debug panel showing GDSync runtime variables: Debug ID, clientID, IAmHost flag, currentLobbyID, and Players count 96

6.1 GDSync's Protected Mode toggle in the plugin settings. When enabled (default), it blocks remote calls on Nodes that haven't been explicitly exposed via `GDSync.expose_func()` or `GDSync.expose_var()` 101

List of Tables

2.1 HPC concepts mapped to game mechanics 34

3.1 Game engine comparison for HPC Sorting Game 51
3.2 Systematic mapping of HPC concepts to game mechanics 57

7.1 Implemented pedagogical features 114
7.2 Frame rate performance by browser 115
7.3 Memory usage by game state 115
7.4 Browser compatibility 116
7.5 Perceived advantages over traditional methods 121
7.6 Requirements fulfillment status 122

D.1 Detailed game-to-OpenMP concept mapping 167

Listings

2.1 Simple OpenMP parallel loop . . . 29
2.2 Simple MPI message passing . . . 29
4.1 Main menu scene structure (simplified) . . . 62
4.2 Single-player scene hierarchy . . . 63
4.3 Lobby scene structure . . . 66
4.4 Multiplayer scene hierarchy . . . 67
4.5 Card component class definition . . . 68
4.6 Card drag-and-drop logic . . . 69
4.7 CardManager class structure . . . 70
4.8 CardManager core methods . . . 70
4.9 MultiplayerCardManager inheritance . . . 72
4.10 Key synchronization methods . . . 73
4.11 Visibility tracking . . . 74
4.12 Scrollable card container structure . . . 75
4.13 Drop slot calculation . . . 75
4.14 GDSync setup in MultiplayerCardManager . . . 77
4.15 Using GDSync RPCs . . . 78
4.16 Handling player disconnections . . . 79
5.1 Project directory structure . . . 89
5.2 Card component structure (card.gd) . . . 90
5.3 Sorting validation algorithm . . . 92
5.4 GDSync node configuration . . . 93
5.5 Card visibility control . . . 94
6.1 Card order synchronization approach . . . 104

Chapter 1

Introduction

1.1 Context and Motivation

1.1.1 The Challenge of Teaching High-Performance Computing

High-Performance Computing (HPC) has become an indispensable tool in modern computational science, powering everything from weather forecasting and molecular dynamics simulations to machine learning and big data analytics. As computational problems grow in scale and complexity, the ability to write efficient parallel programs has transitioned from a specialized skill to a fundamental competency for software engineers and computational scientists.

However, teaching parallel computing concepts presents unique pedagogical challenges. Students frequently find it difficult to visualize how multiple threads or processes interact, communicate, and coordinate to solve problems efficiently. The principal challenges include:

- **Abstraction Gap**: Concepts like threads, processes, synchronization, and message passing are inherently abstract and difficult to visualize
- **Cognitive Load**: Understanding parallel algorithms requires simultaneous consideration of multiple execution flows, shared resources, and synchronization primitives
- **Limited Immediate Feedback**: Traditional HPC assignments involve slow compile-submit-debug cycles that impede learning
- **Language Barriers**: HPC programming typically requires C/C++ or Fortran—languages with steep learning curves that can overshadow conceptual understanding. Students often struggle more with template errors and memory management than with parallelism itself due to the focus of C++ on keeping backwards compatibility as first priority
- **High Entry Barrier**: Setting up HPC environments and debugging distributed systems requires significant technical expertise that distracts from core concepts

Traditional lecture-based approaches struggle to convey the dynamic, interactive nature of parallel processes, motivating the exploration of alternative pedagogical approaches.

1.1.2 Serious Games as Educational Tools

Serious games—games designed with a primary purpose beyond entertainment—have emerged as powerful educational tools. By leveraging game mechanics such as immediate feedback, progressive challenges, and interactive exploration, serious games can make complex concepts more accessible and engaging. Educational games offer several advantages:

- **Active Learning**: Players learn by doing, not just reading or watching
- **Immediate Feedback**: Instant consequences of actions enable rapid learning cycles
- **Visualize Abstract Concepts**: Game representations make invisible processes visible
- **Safe Experimentation**: Risk-free environment for trial and error without expensive computational resources
- **Lower Entry Barrier**: Abstract away complex setup and language syntax to focus on core concepts

In the context of HPC education, serious games can transform abstract parallelization strategies into tangible, manipulable activities, helping students build intuition before tackling the complexities of parallel programming languages.

1.1.3 From Physical Experiments to Digital Games

The pedagogical approach underlying this thesis stems from classroom experiments conducted by Professor Daniele D'Agostino, where students physically sorted numbered cards to simulate parallel computing paradigms:

- **Sequential Execution**: A single student sorts all cards alone—demonstrating traditional sequential algorithms
- **OpenMP Simulation**: Multiple students work collaboratively at a shared desk with all cards visible in a common container, using private buffer areas for local sorting—demonstrating shared memory parallelism with thread-local storage

These physical activities were highly effective at conveying parallel computing concepts in a tangible, memorable way. However, they faced significant limitations: scalability (limited by classroom size), repeatability (difficult to reproduce), and accessibility (requires physical presence).

This thesis aims to capture that pedagogical effectiveness in a scalable, digital format accessible via web browsers, enabling students worldwide to experience similar learning benefits without the constraints of physical classrooms.

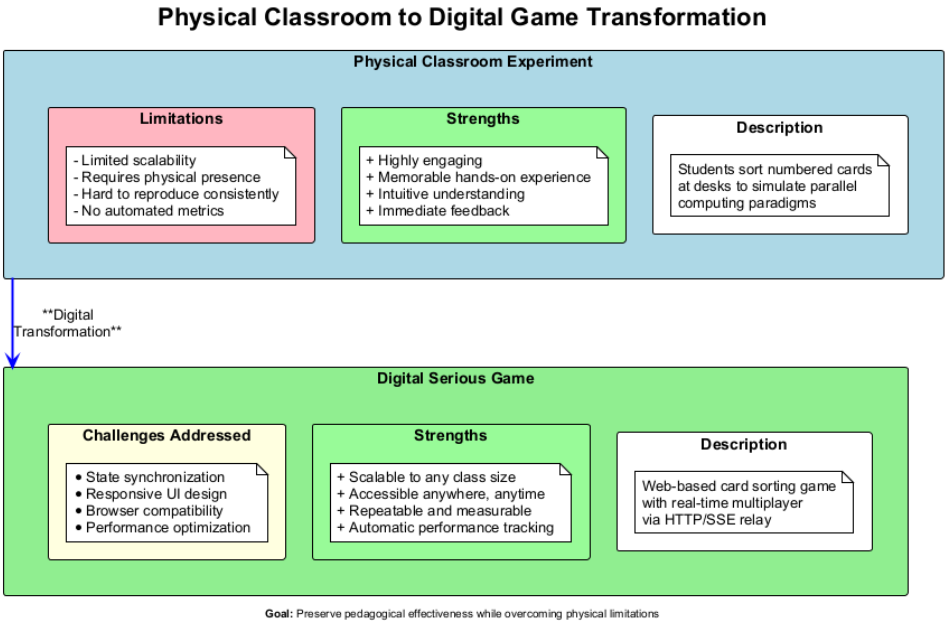


Figure 1.1: Transformation from physical classroom experiments to digital serious game, preserving pedagogical effectiveness while overcoming physical limitations

1.2 Problem Statement

This thesis addresses the following research problem:

> *How can we design and implement an effective serious game for web platforms that teaches fundamental High-Performance Computing concepts (specifically OpenMP shared-memory parallelism and sequential vs. parallel execution) through interactive card-sorting gameplay while overcoming the technical challenges of multiplayer state synchronization and responsive UI/UX constraints?*

This overarching problem decomposes into three interrelated challenges:

- **Educational Challenge**: How can abstract parallel computing concepts be mapped to concrete, understandable game mechanics that accurately represent OpenMP paradigms?

- **Technical Challenge**: How can we implement real-time multiplayer gameplay across diverse devices with acceptable latency and consistent state synchronization?
- **Design Challenge**: How can we balance educational effectiveness with engagement and playability across different screen sizes and devices?

Detailed requirements analysis appears in Chapter 3.

1.3 Research Objectives

The primary objective of this thesis is to develop a functional serious game prototype that demonstrates the feasibility of teaching HPC concepts through web-based gaming, creating an open-source platform suitable for future educational research and classroom deployment.

1.3.1 Core Objectives

1. **Develop an Interactive Serious Game**: Create a game using card sorting as a metaphor for parallel sorting algorithms, accurately mapping game mechanics to OpenMP paradigms
2. **Implement Web-First Functionality**: Develop the game as a web application that runs in browsers with responsive design across different screen sizes and devices
3. **Create Multiplayer Capability**: Implement real-time multiplayer functionality enabling 2–4 players to collaborate, simulating distributed computing scenarios
4. **Evaluate Technical Feasibility**: Assess challenges and solutions for web-based multiplayer game development, particularly regarding state synchronization using the GDSync framework and HTTP/SSE relay architecture
5. **Document Development Process**: Provide comprehensive documentation of technology choices, implementation challenges, and solutions to guide future developers of educational games

1.4 Proposed Solution: HPC Sorting Serious Game

This thesis presents the **HPC Sorting Serious Game**, a web-first educational game that teaches parallel computing through interactive card-sorting. The game uses a simple yet effective metaphor: sorting numbered cards represents sorting data in parallel computing systems. By manipulating virtual cards, students gain

hands-on experience with parallelization strategies without the complexity of actual parallel programming.

1.4.1 Core Concept

The game presents players with a deck of numbered cards that must be sorted in ascending order. This simple metaphor maps directly to parallel sorting algorithms, with different game modes representing different execution paradigms:

Single-Player Mode (Sequential Execution): Represents traditional sequential algorithm execution where a single process sorts all data. The player works alone with all cards visible, performing the entire sorting task independently.

Multiplayer Mode (OpenMP Shared-Memory Parallelism): Represents shared-memory parallelism where multiple threads cooperate on a single address space. All players work collaboratively with cards visible in a shared container (representing shared memory), each player uses private buffer zones for local sorting (representing thread-local storage), and players work simultaneously without forced turn-taking (representing OpenMP's implicit synchronization and parallel execution model).

The complete architecture and pedagogical mapping are detailed in Chapters 3 and 4.

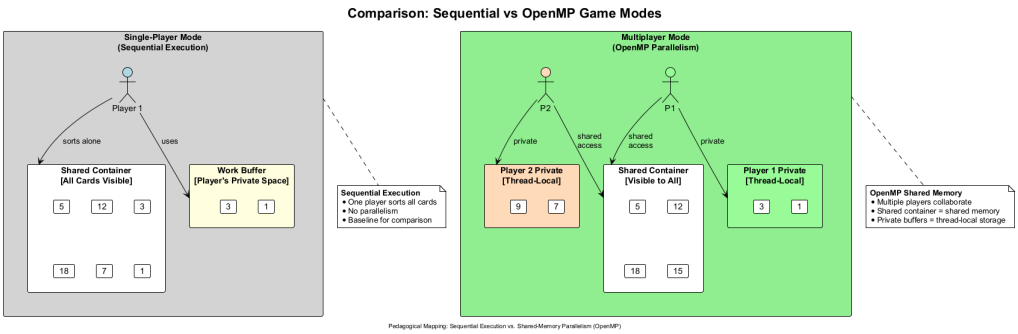


Figure 1.2: Comparison of game modes: Single-player represents sequential execution (left) where one player sorts all cards independently; Multiplayer represents OpenMP shared-memory parallelism (right) where multiple players collaborate with a shared container and private buffers

1.4.2 Technology Stack

The game is built using open-source technologies:

- **Godot Engine 4.x**: Lightweight game engine with excellent 2D capabilities and web export support

- **GDScript**: Python-like scripting language for rapid development
- **GDSync Framework**: Multiplayer state synchronization framework
- **HTTP/SSE Relay Server**: Custom server for real-time web-based multiplayer communication

The rationale for these technology choices is detailed in Chapter 3.

1.5 Thesis Contributions

This thesis makes four primary contributions to the fields of HPC education, serious game development, and web-based multiplayer architecture:

1. **Pedagogical Innovation**: A novel game-based approach for teaching OpenMP shared-memory parallelism and sequential vs. parallel execution comparison, directly inspired by successful physical classroom experiments, with systematic mapping between game mechanics and parallel computing paradigms. Additionally, demonstrates that fundamental HPC concepts can be taught effectively without requiring students to first master complex programming languages like C/C++, reducing cognitive load.
2. **Technical Implementation**: A functional web-based serious game demonstrating real-time multiplayer synchronization for OpenMP simulation, responsive UI/UX for displaying 50+ interactive elements across diverse screen sizes, and solutions to web platform constraints including state consistency and network latency management.
3. **Comprehensive Documentation**: Detailed analysis of technical challenges (particularly GDSync framework issues, multiplayer synchronization, and web browser constraints) and solutions, providing practical guidance for educational game developers.
4. **Open-Source Platform**: A freely available, extensible system for HPC education research, suitable for future academic publications and immediate classroom deployment.

Detailed discussion of these contributions and their implications appears in Chapter 8.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

Chapter 2 reviews parallel computing fundamentals (focusing on OpenMP shared-memory parallelism), serious games in education, web game development challenges, and multiplayer architecture patterns, identifying gaps in existing HPC educational tools.

Chapter 3 describes the Design Science Research methodology, requirements analysis, and technology selection decisions.

Chapter 4 presents the system design and architecture, including scene structure, component design, multiplayer networking, and responsive UI/UX considerations.

Chapter 5 details the technical implementation, including development environment, single-player and multiplayer implementation, and web-specific optimizations.

Chapter 6 provides an honest analysis of difficulties encountered during development, including GDSync framework issues, multiplayer synchronization challenges, and web browser constraints.

Chapter 7 presents completed system features, technical performance metrics, platform compatibility assessment, and preliminary evaluation of educational effectiveness.

Chapter 8 summarizes contributions, reflects on research questions, discusses implications, and outlines future work directions.

Appendices provide supplementary materials including user manual, code listings, API documentation, and study materials.

Chapter 2

Background and Literature Review

This chapter provides the foundational knowledge and contextual background necessary to understand the HPC Sorting Serious Game. We review key concepts in parallel computing, serious games in education, web-based game development, and multiplayer networking architectures.

2.1 Parallel Computing Fundamentals

Parallel computing is the simultaneous execution of multiple computational tasks to solve problems more efficiently than sequential processing. As Moore's Law approaches its physical limits, parallel computing has become essential for improving computational performance across scientific computing, data analytics, and real-time systems.

2.1.1 Parallel Programming Paradigms

Two primary paradigms dominate parallel computing education and practice: shared-memory parallelism and distributed-memory parallelism.

2.1.1.1 Shared-Memory Parallelism

In shared-memory systems, multiple processors or cores access a common address space. All threads can read and write to the same memory locations, enabling efficient data sharing but requiring careful synchronization to prevent race conditions.

Characteristics:

- All processors access the same physical memory
- Communication occurs through shared variables
- Synchronization primitives (locks, barriers, atomic operations) prevent conflicts
- Lower communication overhead compared to distributed memory

- Limited scalability due to memory bandwidth constraints

Applications: Shared-memory parallelism is ideal for problems requiring frequent communication between processing units, such as iterative algorithms, graph processing, and data analytics on multicore processors.

2.1.1.2 Distributed-Memory Parallelism

In distributed-memory systems, each processor has its own local memory. Processors communicate by explicitly sending and receiving messages over a network, requiring programmers to manage data distribution and communication patterns.

Characteristics:

- Each processor has private memory space
- Communication requires explicit message passing
- Scalable to thousands or millions of processors
- Higher communication overhead and latency
- No shared state eliminates race conditions but complicates programming

Applications: Distributed-memory parallelism is essential for large-scale scientific simulations, weather modeling, molecular dynamics, and big data processing across cluster and supercomputer architectures.

2.1.2 OpenMP: Shared-Memory Programming

OpenMP (Open Multi-Processing) is a widely-used API for shared-memory parallel programming in C, C++, and Fortran. It uses compiler directives (pragmas) to specify parallel regions, enabling incremental parallelization of sequential code.

Core Concepts:

Parallel Regions Code sections executed by multiple threads simultaneously

Work-Sharing Constructs Distribute loop iterations or tasks among threads

Data Scoping Variables can be shared (visible to all threads) or private (thread-local copies)

Synchronization Barriers, critical sections, and atomic operations coordinate thread execution

Scheduling Strategies (static, dynamic, guided) control how iterations are assigned to threads

```
#pragma omp parallel for schedule(dynamic)
for (int i = 0; i < N; i++) {
    array[i] = compute_value(i);
}
```

Listing 2.1: Simple OpenMP parallel loop

Educational Value: OpenMP's simplicity makes it excellent for teaching parallel programming concepts. The incremental approach—starting with sequential code and adding pragmas—allows students to observe immediate performance improvements while learning about thread management, data dependencies, and race conditions.

2.1.3 MPI: Distributed-Memory Programming

Example Code:

> **Note:** This section provides background on MPI for completeness and context within the broader HPC landscape. **The game implemented in this thesis focuses exclusively on OpenMP shared-memory parallelism.** MPI distributed-memory concepts are discussed here for educational context but are **not implemented** in the current version. See Chapter 8 for discussion of MPI as potential future work.

MPI (Message Passing Interface) is the de facto standard for distributed-memory parallel programming. It provides a rich set of communication primitives for sending and receiving messages between processes, enabling scalable parallel computing on clusters and supercomputers.

Core Operations:

Point-to-Point Communication `MPI_Send` and `MPI_Recv` exchange messages between specific processes

Collective Communication Operations like `MPI_Bcast`, `MPI_Gather`, and `MPI_Reduce` involve all processes in a communicator

Process Groups Communicators organize processes and define communication contexts

Data Types MPI supports both primitive and user-defined data types for message content

```
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```

if (rank == 0) {
    // Master process
    MPI_Send(data, count, MPI_INT, 1, 0, MPI_COMM_WORLD);
} else if (rank == 1) {
    // Worker process
    MPI_Recv(data, count, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
}

MPI_Finalize();
```

Listing 2.2: Simple MPI message passing

Educational Value: MPI teaches students about distributed computing challenges including data distribution, explicit communication, latency management, and scalability. The message-passing model forces programmers to think carefully about data ownership and communication patterns, skills essential for modern distributed systems.

2.1.4 Parallel Sorting Algorithms

Example Code: Sorting is a fundamental algorithmic problem that benefits significantly from parallelization. Several parallel sorting strategies exist, each with different communication and synchronization requirements.

Parallel Merge Sort:

- Recursively divide array into subarrays
- Sort subarrays in parallel
- Merge sorted subarrays (potentially in parallel)
- Well-suited for shared-memory systems
- Time complexity: $O(\frac{n \log n}{p})$ with p processors

Sample Sort (MPI Pattern):

- Distribute data across processes
- Each process sorts local data
- Select sample elements to determine global pivots
- Redistribute data based on pivots
- Each process sorts its final partition

- Commonly used in distributed-memory systems
- *Note: Not implemented in this thesis; discussed for context*

Relevance to Educational Game: The card-sorting game mechanics map to parallel sorting algorithms. In the implemented game, players act as threads (OpenMP) coordinating through shared memory to sort cards efficiently, making abstract algorithmic concepts tangible through gameplay. Future extensions could explore MPI-style distributed sorting (see Chapter 8).

2.2 Serious Games in Education

Serious games—games designed primarily for purposes beyond entertainment—have emerged as powerful educational tools. This section reviews the theoretical foundations and empirical evidence supporting game-based learning.

2.2.1 Definition and Characteristics

Zyda [1] defines serious games as games that "do not have entertainment, enjoyment, or fun as their primary purpose." These games leverage game design principles to achieve learning objectives, skill development, or behavioral change.

Key Characteristics:

- **Clear Learning Objectives**: Explicit educational goals aligned with curricula
- **Engaging Mechanics**: Game elements (challenges, rewards, feedback) maintain motivation
- **Active Learning**: Players learn by doing, not passive observation
- **Immediate Feedback**: Instant consequences of actions reinforce learning
- **Safe Experimentation**: Risk-free environment for trial and error
- **Progressive Difficulty**: Challenges scale with player skill development

2.2.2 Theoretical Foundations

Game-based learning draws on several established educational theories:

2.2.2.1 Constructivism

Constructivist theory posits that learners actively construct knowledge through experience rather than passively receiving information. Games naturally support constructivist learning by placing players in problem-solving scenarios where they discover principles through experimentation.

2.2.2.2 Flow Theory

Csikszentmihalyi [2] describes "flow" as the optimal psychological state where challenge level matches skill level, producing deep engagement. Well-designed educational games maintain flow by dynamically adjusting difficulty, keeping learners in the zone of proximal development.

2.2.2.3 Experiential Learning

Kolb's experiential learning cycle—concrete experience, reflective observation, abstract conceptualization, active experimentation—aligns perfectly with game mechanics. Players experience situations, observe outcomes, form hypotheses, and test them iteratively.

2.2.3 Empirical Evidence

Research consistently demonstrates the effectiveness of serious games for learning:

Connolly et al. [3] Meta-Analysis: A systematic literature review of 129 papers found that serious games positively impact learning outcomes across multiple domains. Key findings include:

- Improved knowledge acquisition and retention
- Enhanced perceptual and cognitive skills
- Increased motivation and engagement
- Better affective and motivational outcomes
- Most effective when integrated with traditional instruction

papastergiou2009digital Computer Science Education: A study of digital game-based learning in high school computer science found that students using educational games demonstrated:

- Significantly higher knowledge acquisition

- Greater enjoyment and motivation
- Preference for game-based learning over traditional methods
- Improved attitudes toward computer science

2.2.4 Serious Games in Computer Science

Several successful serious games target computer science education:

CodeCombat:

- Teaches programming through RPG-style gameplay
- Players write real code to control characters
- Supports Python, JavaScript, and other languages
- Used in thousands of classrooms worldwide

Lightbot:

- Introduces programming concepts through puzzle-solving
- Teaches sequencing, loops, and procedures
- No syntax required—visual programming interface
- Suitable for young learners

Parallel Computing Games: Few games specifically target parallel computing education. Notable exceptions include:

- **PARMACS Simulator**: Interactive visualization of parallel algorithms
- **ParSim**: Educational tool for parallel algorithm design
- **CUDA Learning Games**: GPU programming challenges

However, these tools typically focus on visualization rather than hands-on, game-like interaction, and few provide multiplayer experiences.

2.2.5 Serious Games for HPC Education

High-Performance Computing education faces unique challenges that serious games can address:

Why Games for HPC?

1. **Abstract Concepts**: HPC involves invisible processes (threads, messages, synchronization) that are difficult to visualize through static diagrams
2. **Temporal Dynamics**: Parallelism involves timing, race conditions, and coordination that unfold over time—games naturally model dynamic systems
3. **Scale Mismatch**: Real HPC systems involve millions of operations per second; games can slow down and make these concepts observable
4. **Hands-On Experience**: Students learn parallelism best by experiencing coordination challenges firsthand

Mapping HPC Concepts to Game Mechanics: The following mappings enable intuitive understanding of HPC concepts through gameplay (see Table 2.1).

Table 2.1: HPC concepts mapped to game mechanics

HPC Concept	Game Mechanic
Threads/Processes	Players
Shared Memory	Shared game board/container
Thread-Local Storage	Private player inventory
Synchronization	Turn-taking or barriers
Race Conditions	Conflicting simultaneous actions
Speedup	Time comparison (1 player vs. N players)
Communication Overhead	Time to transfer items between players
Load Balancing	Equal distribution of work

Advantages of Game-Based HPC Learning:

- **No Prerequisites**: Students can experience parallel concepts before learning programming syntax
- **Misconception Surfacing**: Games reveal incorrect mental models (e.g., "more threads always faster")
- **Quantifiable Learning**: Performance metrics (time, moves) provide concrete feedback
- **Collaborative Learning**: Multiplayer games naturally create discussion and peer learning
- **Scalable Delivery**: Web-based games reach students regardless of institutional HPC infrastructure

2.3 Web-Based Game Development

Web browsers provide a universal platform for educational game deployment, eliminating installation barriers and enabling access from any device with a modern browser.

2.3.1 Web Platform Advantages

Accessibility: Web games require no installation—students access the game via URL, eliminating software distribution challenges and platform compatibility concerns.

Cross-Platform: A single web build runs on Windows, macOS, Linux, and even mobile browsers, maximizing reach with minimal development overhead.

Instant Updates: Deploying updates is immediate—no app store approval processes or user-initiated updates required.

Low Barrier: Modern browsers include powerful JavaScript engines, WebGL for graphics, and APIs for real-time communication.

2.3.2 Web Development Challenges

Browser Sandboxing: Security restrictions prevent raw socket access, UDP communication, and local peer discovery—complicating multiplayer networking.

Performance Overhead: WebAssembly and JavaScript have overhead compared to native code, though performance is acceptable for 2D games.

Networking Limitations: Traditional peer-to-peer networking (like WebRTC data channels) can be complex to configure and may not work reliably across all network configurations.

2.3.3 Game Engines for Web

Godot Engine emerged as the optimal choice for this project due to:

- Completely free and open-source (MIT license)
- Excellent 2D rendering performance
- Lightweight web exports (smaller than Unity or Unreal)
- GDScript language similar to Python (accessible for students)
- Active community and extensive documentation

- Built-in HTML5 export templates
- Scene-based architecture promotes modular design

2.4 Multiplayer Game Architecture

Multiplayer functionality is essential for simulating OpenMP shared-memory parallelism with multiple players collaborating on shared data. This section reviews common multiplayer architectures and networking technologies.

2.4.1 Client-Server Architecture

Traditional multiplayer games use a client-server model:

Characteristics:

- Dedicated server hosts game state
- Clients send inputs to server
- Server validates actions, updates state, broadcasts to clients
- Authoritative server prevents cheating
- Requires server infrastructure and maintenance

Advantages:

- Strong consistency and cheat prevention
- Clients need not communicate directly
- Server can enforce rules and validate actions

Disadvantages:

- Requires dedicated servers (cost, maintenance)
- Single point of failure
- Latency for all communications
- Scalability challenges with many concurrent games

2.4.2 Peer-to-Peer Architecture

Peer-to-peer (P2P) networks eliminate dedicated servers by allowing clients to communicate directly:

Characteristics:

- Clients connect directly to each other
- No central server required (except for matchmaking)
- One peer often designated as "host" for authority
- Lower latency for direct connections
- More complex connection establishment (NAT traversal)

Advantages:

- No server infrastructure costs
- Lower latency between peers
- Decentralized—no single point of failure
- Scalable for small groups (2–8 players)

Disadvantages:

- NAT traversal complexity
- Host has performance burden
- Cheat prevention more difficult
- Connection quality varies with peer networks

2.4.3 Web Transport Technologies

Web browsers offer several options for real-time communication, each with trade-offs:

HTTP Polling:

- Client repeatedly requests updates from server
- Simple to implement, universally supported
- Higher latency and server load due to frequent requests
- Suitable for low-frequency updates

Server-Sent Events (SSE):

- Server pushes events to client over HTTP
- Simpler than WebSocket (HTTP-based, no special protocol)
- Unidirectional (server to client only)
- Automatic reconnection built into browser API
- Good browser support and proxy compatibility

HTTP + SSE Relay Architecture: For web-based multiplayer games requiring broad browser compatibility, a relay server architecture using HTTP POST for client-to-server communication and SSE for server-to-client events provides:

- Universal browser support without WebSocket complexity
- Works through firewalls and proxies that block WebSocket
- Simplified connection establishment (no STUN/TURN required)
- Trade-off: All traffic routes through server (no P2P)
- Acceptable latency for turn-based or slower-paced games

2.4.4 State Synchronization Patterns

Maintaining consistent game state across multiple clients is challenging. Common patterns include:

Deterministic Lockstep:

- All clients simulate the same game logic
- Inputs synchronized and executed simultaneously
- Guarantees identical state on all clients
- Sensitive to latency and packet loss
- Used in RTS games (Age of Empires, StarCraft)

Client-Side Prediction:

- Clients predict results of actions immediately
- Server validates and corrects if necessary
- Provides responsive feel despite latency
- Requires reconciliation when predictions wrong
- Used in FPS games (Counter-Strike, Overwatch)

Authoritative Server/Host:

- Server/host maintains canonical state
- Clients send actions, receive state updates
- Simple to implement and reason about
- Higher latency but stronger consistency
- Suitable for turn-based or slower-paced games

Selected Approach: For the HPC Sorting Game, a **host-authoritative model** was chosen because:

- Educational focus prioritizes clarity over competitive performance
- Collaborative nature tolerates moderate latency
- Simpler implementation reduces development complexity
- Prevents inconsistencies and ensures fair learning environment
- Aligns with centralized coordination patterns in parallel systems

2.5 Related Work

This section reviews existing educational tools and games related to parallel computing and serious games.

2.5.1 Parallel Computing Educational Tools

Several tools exist for teaching parallel computing, each with strengths and limitations:

PARMACS:

- Parallel algorithm visualization tool
- Shows thread/process execution timelines
- Limited interactivity—primarily observational
- Desktop-only platform

OpenMP Visualization Tools:

- Intel VTune, TAU (Tuning and Analysis Utilities)
- Focused on performance profiling
- Steep learning curve for students
- Not designed for introductory education

MPI Tools:

- MPICH, Open MPI with debugging tools
- Require cluster access or virtual machines
- Focus on actual parallel programming, not conceptual learning
- *Note: MPI concepts discussed for context; not covered in this thesis*

Gap Identification: Existing tools generally fall into two categories:

1. **Visualization tools**: Show parallel execution but lack hands-on interaction
2. **Programming environments**: Require coding skills, deterring beginners

Few tools bridge the gap between conceptual understanding and practical implementation. Fewer still are designed for web platforms or incorporate game mechanics to enhance engagement.

2.5.2 Serious Games for Computing Education

Several serious games successfully teach computing concepts:

CodeCombat:

- Teaches programming through RPG gameplay
- Players write real code in Python/JavaScript
- Excellent for syntax and basic algorithms
- Does not address parallel computing

Human Resource Machine:

- Puzzle game teaching assembly-like programming
- Visual programming—no syntax barriers
- Excellent for teaching sequential thinking
- Single-threaded—no parallelism concepts

Shenzhen I/O:

- Hardware design and programming game
- Teaches low-level optimization
- Complex and targeted at experienced programmers
- No explicit parallel computing focus

Gap in Parallel Computing Games: No widely-adopted serious game specifically teaches parallel computing concepts like OpenMP shared-memory parallelism through interactive, game-based mechanics on web platforms. This gap motivated the development of the HPC Sorting Serious Game, which focuses on OpenMP concepts with potential for future MPI extensions.

2.5.3 Physical Teaching Methods

The pedagogical approach for this thesis draws inspiration from physical classroom activities:

CS Unplugged:

- Collection of unplugged computer science activities
- Teaches algorithms, sorting, searching without computers
- Highly engaging but not scalable or repeatable
- Limited to in-person classroom settings

Professor D'Agostino's Card Sorting Experiments: As described in Chapter 1, Professor Daniele D'Agostino conducted classroom experiments where students physically sorted numbered cards to simulate parallel computing paradigms. These experiments demonstrated:

- High student engagement and retention
- Intuitive understanding of parallelism concepts
- Memorable hands-on experience
- Clear differentiation between sequential and parallel execution models

However, physical experiments have limitations:

- Require physical classroom presence
- Not scalable to large classes
- Difficult to reproduce consistently
- Limited flexibility in parameters (card count, student count)
- No performance metrics or automated feedback

Digital Transformation: This thesis aims to capture the pedagogical effectiveness of physical card-sorting experiments in a digital, web-first serious game that overcomes the limitations of physical activities while preserving their educational benefits.

2.6 Summary

This chapter established the foundational knowledge necessary to understand the HPC Sorting Serious Game:

- **Parallel Computing**: Reviewed OpenMP shared-memory parallelism (the focus of this thesis) and MPI distributed-memory paradigms (for context), highlighting their educational challenges and importance.
- **Serious Games**: Examined theoretical foundations, empirical evidence, and successful examples of game-based learning in computer science education.
- **Web Development**: Discussed advantages and challenges of web platforms for educational games, and the rationale for selecting Godot Engine with web export capabilities.
- **Multiplayer Architecture**: Compared client-server and peer-to-peer models, introduced web transport technologies (HTTP/SSE), and justified the host-authoritative approach for OpenMP simulation.
- **Related Work**: Identified gaps in existing parallel computing educational tools and serious games, motivating the development of this project.

The next chapter describes the research methodology, requirements analysis, and technology selection decisions that guided the development of the HPC Sorting Serious Game.

Chapter 3

Methodology

This chapter describes the research methodology, requirements analysis, design approach, and technology selection decisions that guided the development of the HPC Sorting Serious Game. The methodology follows a Design Science Research approach, emphasizing iterative development and evaluation.

3.1 Research Methodology

3.1.1 Design Science Research Framework

This thesis adopts the Design Science Research (DSR) methodology [4, 5], which is particularly well-suited for developing innovative artifacts that address practical problems while contributing to theoretical knowledge.

DSR Process Model: The DSR process consists of six iterative activities:

1. **Problem Identification and Motivation**: Identify the research problem (teaching HPC concepts) and justify the value of a solution (web-based serious game).
2. **Define Objectives of a Solution**: Infer solution objectives from problem definition and existing knowledge (create engaging, accessible, educational multiplayer game).
3. **Design and Development**: Create the artifact (HPC Sorting Serious Game) by selecting appropriate technologies and implementing game mechanics.
4. **Demonstration**: Demonstrate the artifact's utility by applying it to the problem context (students learning parallel computing).
5. **Evaluation**: Observe and measure how well the artifact supports a solution to the problem (usability testing, performance benchmarking).
6. **Communication**: Communicate the problem, artifact, evaluation, and contributions to relevant audiences (thesis, potential publications).

Rationale for DSR: DSR is appropriate for this research because:

- The goal is to create a functional artifact (serious game) rather than merely analyze existing phenomena
- The problem is practical and relevant to HPC education
- The solution requires integrating knowledge from multiple domains (pedagogy, game design, parallel computing, web development)
- Evaluation can be both utility-based (does it work?) and design-based (is it well-constructed?)

3.1.2 Development Approach

The game development followed an iterative, incremental approach:

Phase 1: Concept and Prototyping (Weeks 1–3)

- Defined core mechanics based on physical card-sorting experiments
- Created paper prototypes and mockups
- Evaluated technology options (game engines, frameworks)
- Developed minimal prototype with basic card rendering

Phase 2: Single-Player Implementation (Weeks 4–7)

- Implemented card generation, shuffling, and rendering
- Developed drag-and-drop mechanics for touch and mouse input
- Created buffer zones for private card storage
- Implemented sorting validation and game completion logic
- Added timer and move counting

Phase 3: Multiplayer Foundation (Weeks 8–12)

- Developed HTTP/SSE relay server for web browser compatibility
- Implemented lobby system for matchmaking
- Developed basic state synchronization
- Integrated GDSync framework with custom web transport layer
- Tested connection establishment and stability

Phase 4: Multiplayer Refinement (Weeks 13–16)

- Implemented card visibility management (private buffers)
- Developed host-authoritative state management
- Added disconnection handling and recovery
- Refined synchronization for card movements
- Addressed framework issues (GDSync protected mode)

Phase 5: Polish and Testing (Weeks 17–20)

- UI/UX improvements for web usability
- Performance optimization (rendering, networking)
- Extensive testing across web browsers
- Documentation and code cleanup
- Preparation for evaluation and thesis writing

3.2 Requirements Analysis

A comprehensive requirements analysis was conducted to ensure the game met educational and functional objectives.

3.2.1 Educational Requirements

The primary purpose of the game is education. Requirements were derived from HPC learning objectives and parallel computing curricula:

ER1: OpenMP Simulation

- The game must simulate shared-memory parallelism
- All players must see all cards in the main container (shared memory)
- Players must have private buffers (thread-local storage)
- No explicit communication required between players (implicit synchronization)

ER2: Sequential Execution Baseline

- The game must provide a single-player mode for sequential execution comparison
- A single player must sort all cards without parallelism
- This mode serves as a baseline for understanding speedup from parallelization
- Players should be able to compare sequential vs. parallel performance

ER3: Performance Feedback

- The game must provide timing information (simulating speedup measurement)
- Move counting must encourage algorithmic efficiency
- Visual feedback must reinforce correct and incorrect actions

ER4: Progressive Difficulty

- Support variable numbers of cards (10–200) for scalability lessons
- Support variable numbers of players (2–4) for parallelism degree
- Provide different sorting challenges (random, partially sorted, reverse sorted)

ER5: Conceptual Clarity

- Game mechanics must clearly map to HPC concepts
- Visual design must distinguish shared vs. private data
- Terminology should align with parallel computing vocabulary when appropriate

3.2.2 Functional Requirements

Functional requirements specify what the system must do:

FR1: Card Management

- Generate configurable numbers of cards with unique values
- Shuffle cards randomly at game start
- Render cards with clear visual representation (number, state)
- Support dragging and dropping cards between containers

FR2: Single-Player Mode

- Provide a single-player game mode representing sequential execution
- Display all cards in a scrollable container
- Provide work buffer zones for temporary storage
- Validate sorting order upon completion
- Display completion time and move count

FR3: Multiplayer Mode

- Enable 2–4 players to connect via server-relayed networking
- Provide lobby for matchmaking and game configuration
- Display shared container visible to all players (OpenMP shared memory)
- Synchronize card movements across all clients
- Provide private buffers for each player (thread-local storage)
- Hide cards in other players' private buffers (thread-private data)
- Handle player disconnections gracefully

FR4: User Interface

- Main menu with navigation to single-player and multiplayer modes
- Lobby interface for creating/joining games
- In-game UI with timer, move counter, player indicators
- Victory screen with performance summary
- Settings for theme, controls, and game parameters

FR5: Feedback Systems

- Toast notifications for events (player joined, card moved, errors)
- Visual animations for card pickup, drop, invalid moves
- Sound effects for actions (optional, can be disabled)
- Confetti or celebration effects upon game completion

FR6: Quality Attributes

- Target 60 FPS in modern web browsers
- Game initialization and card generation under 2 seconds
- Support up to 200 cards without significant performance degradation
- Intuitive interface requiring no tutorial for basic gameplay
- Responsive drag-and-drop with visual feedback
- Graceful handling of network disconnections
- Clean code structure with modular design
- Open-source license (MIT) for educational reuse
- No account creation or authentication required

3.3 Technology Selection

Technology choices significantly impact development efficiency, game performance, and future extensibility. This section justifies the selection of each major technology component.

3.3.1 Game Engine: Godot 4.x

Evaluation Criteria: Game engines were evaluated based on:

- License and cost
- 2D rendering capabilities
- Web platform support (HTML5 export)
- Learning curve and documentation
- Community support and plugin ecosystem
- Multiplayer/networking support
- Performance in browsers

Alternatives Considered:

Table 3.1: Game engine comparison for HPC Sorting Game

Criterion	**Unity**	**Unreal**	**Godot**	**Cocos2d**
Free/Open Source	Partial	Partial	Yes	Yes
2D Performance	Good	Fair	Excellent	Excellent
Web Export	Good	Fair	Excellent	Good
Learning Curve	Medium	Steep	Gentle	Medium
Networking APIs	Good	Excellent	Good	Fair
Community Size	Large	Large	Medium	Small
Export Size	Large	Very Large	Small	Medium

Decision Rationale: Godot Engine was selected because:

1. **Cost**: Completely free and open-source (MIT license), no royalties, subscriptions, or hidden fees. Critical for educational projects with no budget.
2. **2D Optimization**: Godot is specifically optimized for 2D games, unlike Unity and Unreal which prioritize 3D. This results in better performance and smaller export sizes.
3. **Lightweight**: Web export builds are significantly smaller (10–20 MB) compared to Unity (50–100 MB) or Unreal (100+ MB), important for fast loading in browsers.
4. **GDScript**: Python-like scripting language is accessible to students who may examine the source code. Lower barrier to entry than C# (Unity) or C++ (Unreal).
5. **Scene Architecture**: Godot's scene-based architecture promotes modular design and code reuse, aligning well with software engineering best practices.
6. **Built-in Multiplayer**: High-level networking APIs available, though third-party frameworks (GDSync) and custom transport layers were needed for web browser compatibility.
7. **Active Community**: Growing community with extensive tutorials, documentation, and plugins. Version 4.x introduced significant improvements over 3.x.
8. **Ethical Alignment**: Open-source philosophy aligns with educational values and ensures long-term availability without corporate dependencies.

Trade-offs: Godot's limitations include:

- Smaller community and fewer third-party assets compared to Unity
- Fewer commercial games as reference implementations

- Some plugins less mature than Unity Asset Store equivalents
- 3D capabilities lag behind Unity/Unreal (not relevant for this project)

3.3.2 Programming Language: GDScript

Language Options in Godot: Godot supports three primary programming approaches:

1. **GDScript**: Native scripting language, Python-like syntax
2. **C#**: .NET support via Mono runtime
3. **C++**: Native modules via GDNative/GDExtension

Decision Rationale: GDScript was chosen because:

- **Integration**: Tight integration with Godot Engine, optimized for game scripting
- **Simplicity**: Python-like syntax is readable and approachable for students
- **Rapid Development**: Dynamic typing and concise syntax accelerate prototyping
- **Documentation**: Most Godot tutorials and examples use GDScript
- **Performance**: Sufficient for 2D game logic; bottlenecks typically in rendering, not scripting
- **No Build Step**: Interpreted language—no compilation required for iteration

Performance Considerations: GDScript is slower than C++ but acceptable for this use case because:

- Card sorting logic is not computationally intensive
- Godot's rendering engine (written in C++) handles performance-critical operations
- Network latency dominates over local computation time
- Development speed more important than raw execution speed

3.3.3 Multiplayer Framework: GDSync

Multiplayer Implementation Options: Three approaches were considered for multiplayer networking:

1. **Raw Godot Networking**: Use built-in `ENetMultiplayerPeer` and RPC system
2. **GDSync Framework**: Third-party addon providing high-level synchronization abstractions
3. **Custom Protocol**: Build custom WebRTC integration from scratch

Decision Rationale: GDSync was selected because:

- **Abstraction Level**: Provides higher-level abstractions than raw Godot networking, reducing boilerplate code
- **State Synchronization**: Built-in support for synchronizing node properties automatically
- **RPC Simplification**: Cleaner RPC syntax and management compared to Godot's built-in system
- **Active Development**: Maintained project with community support
- **Documentation**: Adequate documentation and examples for common use cases
- **Compatibility**: Works well with Godot 4.x and WebRTC connections

Challenges Encountered: GDSync introduced some difficulties:

- **Protected Mode Issue**: Default "protected" mode blocked RPC calls between peers, requiring workarounds (see Chapter 6)
- **Documentation Gaps**: Some features underdocumented or unclear
- **Debugging**: Opaque error messages complicated troubleshooting
- **Framework Overhead**: Additional layer of abstraction sometimes obscured underlying networking behavior

Despite these challenges, GDSync provided net benefits by significantly reducing the amount of low-level networking code required.

3.3.4 Networking Technology: HTTP/SSE Relay

Web Browser Constraints: Web browsers impose significant restrictions on networking that prevented use of traditional approaches:

- **No UDP Access**: Browsers cannot open raw UDP sockets, eliminating ENet and similar protocols
- **WebRTC Complexity**: While browsers support WebRTC, GDSync's native WebRTC implementation does not function in browser-exported Godot games due to sandboxing
- **No LAN Discovery**: Browser security prevents local network peer discovery

Solution: HTTP + Server-Sent Events (SSE) To overcome browser limitations, a custom relay server architecture was developed:

1. **HTTP POST for Commands**: All game packets sent via standard HTTP POST requests
2. **SSE for Real-time Events**: Server-Sent Events provide push notifications to clients
3. **Server Relay**: All communication passes through the relay server rather than peer-to-peer
4. **Star Topology**: Clients connect to server; host receives, processes, and broadcasts via server

Relay Server Implementation: The relay server (Python/aiohttp) provides:

- `POST /api/lobby/connect` — Client registration and peer ID assignment
- `POST /api/lobby/create` — Create lobby with room code
- `POST /api/lobby/join` — Join existing lobby by code
- `POST /api/lobby/broadcast` — Send game packets to other peers
- `GET /api/lobby/events` — SSE stream for receiving real-time events

Trade-offs: This architecture has trade-offs compared to peer-to-peer approaches:

- **Advantage**: Universal browser compatibility without plugins
- **Advantage**: Simplified connection establishment (no STUN/TURN configuration)

- **Trade-off**: Requires running relay server infrastructure
- **Trade-off**: Higher latency due to server hop (acceptable for turn-based card game)

3.3.5 Supporting Plugins and Tools

Several Godot plugins were integrated to enhance development:

ToastParty:

- Purpose: Display non-intrusive notification messages
- Use: Player joined/left, card moved, errors, game events
- Benefit: Provides feedback without blocking gameplay

Logger:

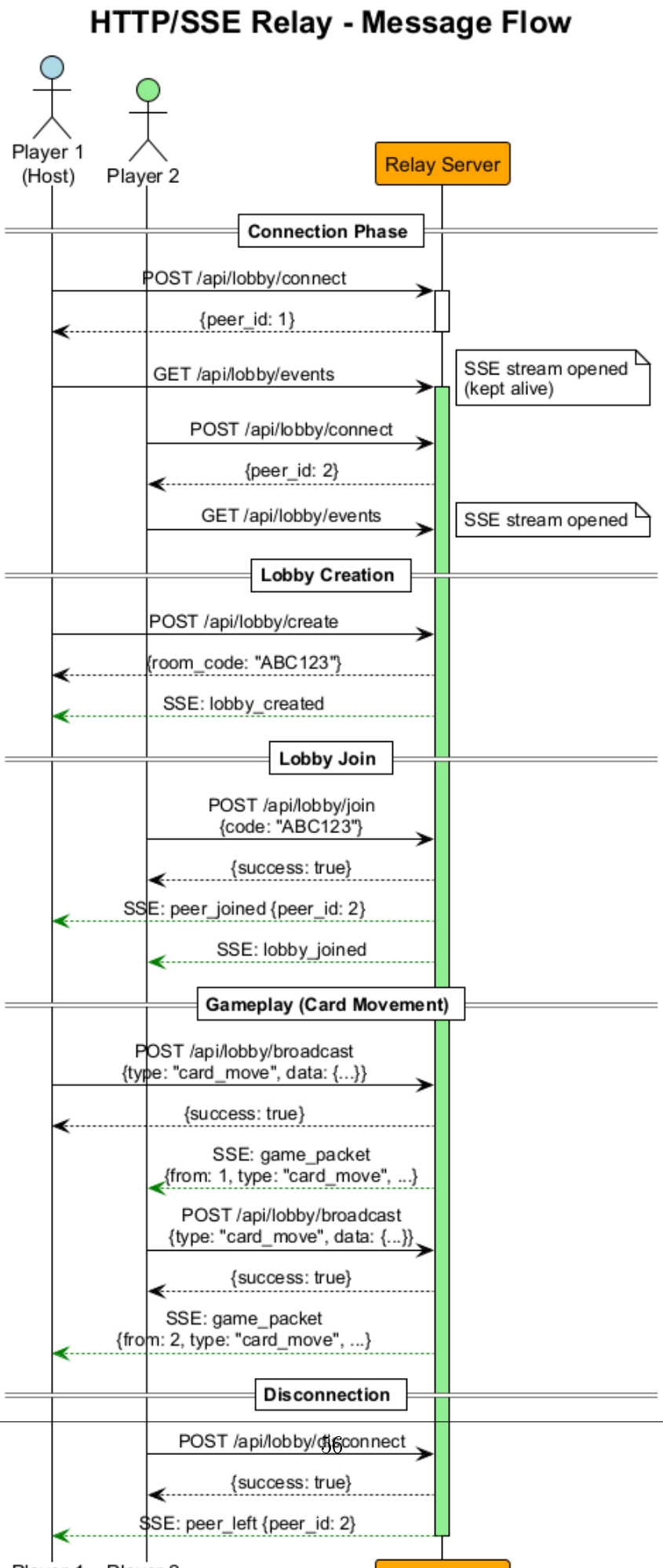
- Purpose: Enhanced debugging output with categorization
- Use: Development troubleshooting and performance analysis
- Benefit: Structured logging with filtering capabilities

VarTree:

- Purpose: Runtime variable inspection and modification
- Use: Debugging game state during development
- Benefit: Observe and manipulate variables without recompiling

Scene-Selector:

- Purpose: Quick scene navigation during development
- Use: Switch between game scenes without main menu navigation
- Benefit: Faster iteration during development


HTTP/SSE Relay - Message Flow
Player 1
(Host)
Player 2
Relay Server
Connection Phase
POST /api/lobby/connect
{peer_id: 1}
GET /api/lobby/events
SSE stream opened
(kept alive)
POST /api/lobby/connect
{peer_id: 2}
GET /api/lobby/events
SSE stream opened
Lobby Creation
POST /api/lobby/create
{room_code: "ABC123"}
SSE: lobby_created
Lobby Join
POST /api/lobby/join
{code: "ABC123"}
{success: true}
SSE: peer_joined {peer_id: 2}
SSE: lobby_joined
Gameplay (Card Movement)
POST /api/lobby/broadcast
{type: "card_move", data: {...}}
{success: true}
SSE: game_packet
{from: 1, type: "card_move", ...}
POST /api/lobby/broadcast
{type: "card_move", data: {...}}
{success: true}
SSE: game_packet
{from: 2, type: "card_move", ...}
Disconnection
POST /api/lobby/disconnect
{success: true}
SSE: peer_left {peer_id: 2}

3.4 Game Design Methodology

3.4.1 Mapping HPC Concepts to Game Mechanics

The core challenge was translating abstract parallel computing concepts into tangible game mechanics. Table 3.2 shows the systematic mapping process:

Table 3.2: Systematic mapping of HPC concepts to game mechanics

HPC Concept	Real-World Analogy	Game Mechanic
Sequential Execution	Single student working alone	Single player in solo mode
Thread (OpenMP)	Student with access to shared desk	Player in multiplayer mode
Shared Memory	Desk visible to all students	Shared card container
Thread-Local Storage	Student's private notebook	Player's private buffer zones
Parallel Execution	Students work simultaneously	Multiple players, no turn-taking
Synchronization	Students coordinate access	Shared container access patterns
Memory Access Overhead	Time to reach shared desk	Network latency to shared data
Speedup	Multiple students finish faster	Timer comparison: solo vs. multi
Load Balancing	Even distribution of work	Fair work distribution among players

Design Validation: The pedagogical mapping was validated through:

- Comparison with Professor D'Agostino's physical experiments
- Review by HPC instructors for conceptual accuracy
- Pilot testing with students familiar with parallel computing
- Iterative refinement based on feedback

3.4.2 User Experience Design

Design Principles:

1. **Minimize Cognitive Load**: Simple, clear visuals; avoid information overload
2. **Immediate Feedback**: Every action produces visual/audio response
3. **Progressive Disclosure**: Show information when needed, hide complexity initially
4. **Error Prevention**: Design interface to prevent common mistakes
5. **Aesthetic Simplicity**: Clean, uncluttered design focuses attention on cards

Interaction Design: Card interactions were designed to work with both touch and mouse input:

- **Click/Tap to Select**: Quick click or tap selects a card (alternative to drag)
- **Drag to Move**: Click-and-hold initiates drag, release drops card
- **Scroll**: Vertical scroll moves through card container
- **Visual Feedback**: Cards scale up when dragged, drop zones highlight

Color Coding: Colors convey semantic meaning:

- **Blue**: Normal cards, main container
- **Yellow**: Highlighted cards during drag
- **Gray**: Disabled/hidden cards (other players' buffers)
- **Purple**: Player-specific UI elements (buffers, indicators)

3.5 Evaluation Methodology

Evaluation Criteria: The game's success was evaluated across multiple dimensions:

1. **Technical Performance**: Frame rate, memory usage, browser compatibility
2. **Functional Completeness**: Implementation of all required features
3. **Usability**: Ease of use, learnability, satisfaction
4. **Educational Effectiveness**: Conceptual understanding, engagement, learning outcomes

Evaluation Methods:

3.5.0.0.0.1 Technical Benchmarking:

- Performance profiling using Godot's built-in tools
- Frame rate monitoring in target browsers
- Memory usage tracking during extended gameplay

3.5.0.0.0.2 Usability Testing:

- Think-aloud protocols with test users
- Observation of first-time players
- Task completion rates and times
- System Usability Scale (SUS) questionnaire (if time permits)

3.5.0.0.0.3 Educational Assessment:

- Pre/post-test knowledge assessments (if formal study conducted)
- Concept mapping to evaluate mental model formation
- Comparison with traditional teaching methods (if feasible)
- Qualitative feedback on perceived learning value

3.6 Development Tools and Environment

Development Platform:

- **OS**: Windows 11 / Linux (Ubuntu 22.04)
- **IDE**: Visual Studio Code with GDScript extension
- **Engine**: Godot Engine 4.2.x
- **Version Control**: Git with GitHub repository

Testing Environment:

- Primary: Chrome 120+ (Windows, macOS)
- Secondary: Firefox 120+ (Windows, Linux)
- Tertiary: Safari 17+ (macOS)
- Local development server for testing

Build and Deployment:

- Web export (HTML5) built via Godot export templates
- Debug builds for development testing
- Release builds with optimizations for distribution
- Static file hosting for deployment

3.7 Summary

This chapter described the comprehensive methodology employed to develop the HPC Sorting Serious Game:

- **Research Approach**: Design Science Research framework with iterative development
- **Requirements**: Educational and functional requirements derived from learning objectives
- **Technology Selection**: Justified choices of Godot Engine, GDScript, GDSync, and HTTP/SSE relay architecture based on project needs
- **Game Design**: Systematic mapping of HPC concepts to game mechanics with UX design principles
- **Evaluation**: Multi-faceted evaluation approach combining technical, usability, and educational assessment

The next chapter presents the detailed system architecture resulting from these methodological decisions, including scene structure, component design, and multiplayer networking architecture.

Chapter 4

System Design and Architecture

This chapter describes the comprehensive design and architecture of the HPC Sorting Serious Game. We present the system from multiple perspectives: high-level scene structure, core component design, multiplayer networking architecture, responsive UI considerations, and detailed data flow patterns.

4.1 Architectural Overview

The game follows a component-based architecture typical of modern game engines, with clear separation of concerns between presentation, game logic, and networking layers. The architecture is designed to support both single-player (sequential execution) and multiplayer (OpenMP shared-memory simulation) modes with significant code reuse.

4.1.1 Design Principles

The architecture adheres to several key design principles:

1. **Component-Based Design**: Game entities (cards, containers, buffers) are modular components that can be composed and reused.
2. **Separation of Concerns**: Clear boundaries exist between UI presentation, game logic, and networking code.
3. **Signal-Driven Communication**: Components communicate through Godot's signal system rather than tight coupling, improving maintainability.
4. **Host-Authoritative Multiplayer**: One client (the host) serves as the authoritative source of truth to prevent cheating and simplify conflict resolution.
5. **Responsive UI**: All interfaces are designed with responsive layouts that adapt to different screen sizes and support both touch and mouse input.
6. **State Synchronization**: Clear patterns for synchronizing game state across multiple clients using the GDSync framework.

4.1.2 System Layers

The system is organized into distinct logical layers:

Presentation Layer Handles all visual rendering, animations, user input (touch/-mouse events), and UI components (menus, dialogs, toasts).

Game Logic Layer Implements core game mechanics including card sorting validation, buffer management, timer control, and game state transitions.

Networking Layer Manages relay server connections via HTTP/SSE, state synchronization through GDSync, lobby system for player matchmaking, and host election.

Framework Layer Provides reusable utilities including scene management, settings persistence, theme management, logging, and debug tools.

4.2 Scene Structure

The game is organized into several distinct scenes, each representing a major screen or functional area. Godot's scene-based architecture naturally supports this modular organization.

4.2.1 Main Menu Scene

The entry point of the application, providing navigation to all major features.

Responsibilities:

- Display game title and branding
- Provide navigation buttons to single-player and multiplayer modes
- Access to settings (theme, audio, controls)
- Exit game or close browser tab

```
MainMenuScene (Control)
|-- BackgroundPanel (Panel)
|-- TitleLabel (Label) "HPC Sorting Game"
|-- ButtonContainer (VBoxContainer)
|    |-- SinglePlayerButton (Button)
|    |-- MultiplayerButton (Button)
|    |-- SettingsButton (Button)
|    +-- QuitButton (Button)
```

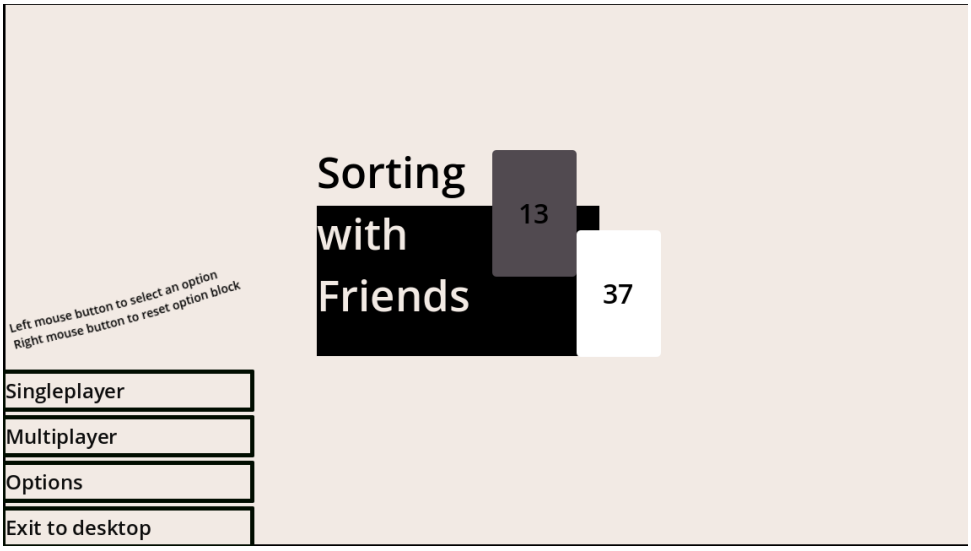


Figure 4.1: Main menu interface showing the game title, mode selection buttons, and animated card decorations

```
+-- VersionLabel (Label)
```

Listing 4.1: Main menu scene structure (simplified)

4.2.2 Single-Player Game Scene

Key Components: Implements the OpenMP simulation gameplay where players sort cards using private buffers.

Responsibilities:

- Display all cards in a shared scrollable container
- Provide private buffer zones for local sorting
- Track game timer and move count
- Validate sorting order and detect completion
- Show victory screen with performance metrics

```
SinglePlayerScene (Control)
|-- CardManager (Node) [Script: card_manager.gd]
|    |-- Timer (Timer)
|    +-- GameState (Dictionary)
|-- UI (Control)
|    |-- TopBar (HBoxContainer)
|    |    |-- TimerLabel (Label)
```

```
|    |    |-- MoveCountLabel (Label)
|    |    +-- BackButton (Button)
|    |-- CardScrollContainer (ScrollContainer)
|    |    +-- CardFlowContainer (HFlowContainer)
|    |        +-- [Card instances added dynamically]
|    +-- BufferContainer (VBoxContainer)
|        |-- Buffer1 (Panel)
|        |-- Buffer2 (Panel)
|        +-- Buffer3 (Panel)
+-- FinishGameWindow (PopupPanel)
    |-- CongratsLabel (Label)
    |-- TimeLabel (Label)
    |-- MovesLabel (Label)
    |-- RestartButton (Button)
    +-- MenuButton (Button)
```

Listing 4.2: Single-player scene hierarchy

Key Design Decisions:

- **ScrollContainer for Cards**: Enables viewing large numbers of cards (50–200) on screens of various sizes by providing vertical scrolling.
- **FlowContainer Layout**: Automatically wraps cards to multiple rows based on screen width, providing responsive layout.
- **Separate Buffer Zones**: Physically distinct areas on screen to clearly represent private storage separate from shared container.
- **Single CardManager Node**: Centralized logic for game state, validation, and event coordination.

4.2.3 Multiplayer Lobby Scene

Handles player matchmaking, room creation/joining, and connection management before gameplay.

Responsibilities:

- Create or join game rooms via unique room codes
- Display list of connected players
- Handle host election and role assignment
- Configure game settings (number of cards, rules)

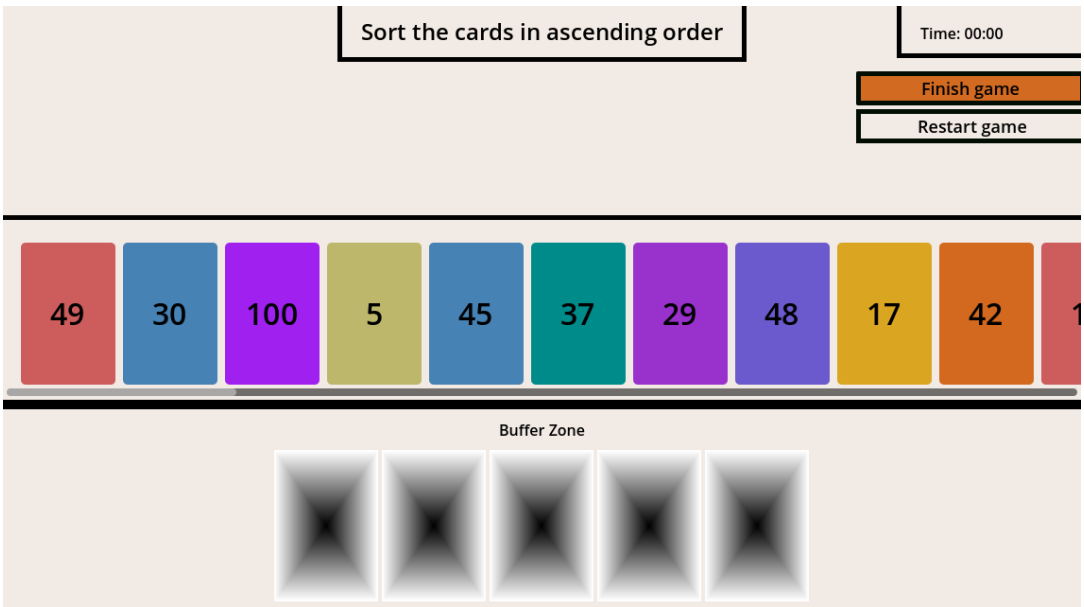


Figure 4.2: Single-player game interface showing numbered cards in the shared container and empty buffer zones below

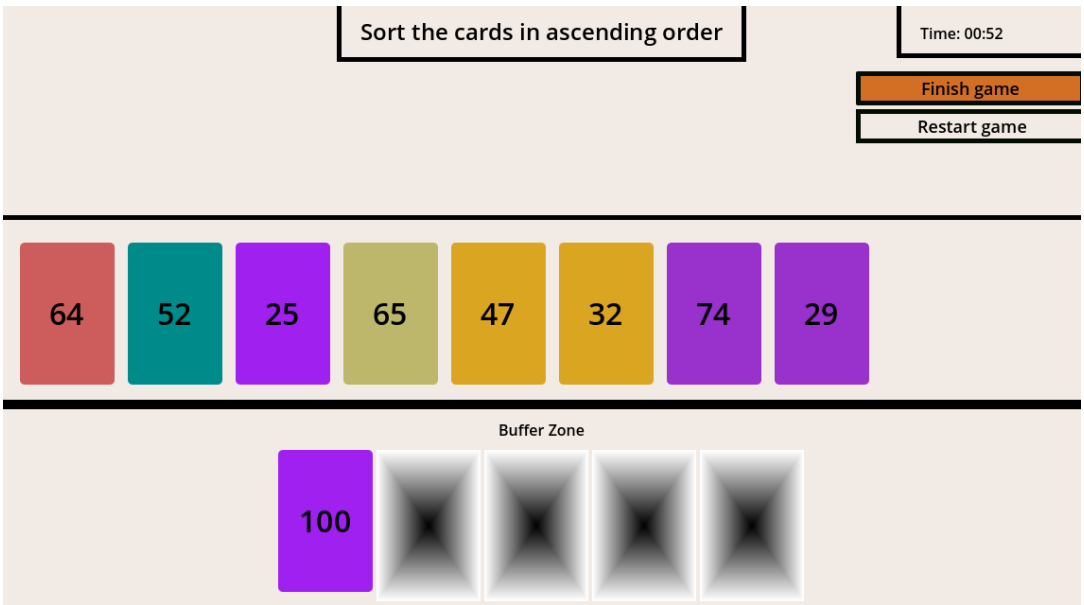


Figure 4.3: Single-player game with a card (100) placed in the buffer zone, demonstrating thread-local storage simulation

- Start game when all players are ready
- Manage connection status and timeouts

```
LobbyScene (Control)
|-- ConnectionManager (Node) [Script: connection_manager.gd]
|   |-- HTTPSSEConnection (Node)
|   |-- SignalingClient (WebSocketClient)
|   +-- PeerConnections (Dictionary)
|-- UI (Control)
|   |-- CreateRoomPanel (Panel)
|   |   |-- RoomCodeLabel (Label)
|   |   |-- CopyCodeButton (Button)
|   |   +-- StartGameButton (Button)
|   |-- JoinRoomPanel (Panel)
|   |   |-- RoomCodeInput (LineEdit)
|   |   +-- JoinButton (Button)
|   |-- PlayerListContainer (VBoxContainer)
|   |   +-- [PlayerCard instances]
|   +-- SettingsPanel (Panel)
|       |-- CardCountSlider (HSlider)
|       +-- ReadyCheckbox (CheckBox)
+-- ToastParty (Node) [Notifications]
```

Listing 4.3: Lobby scene structure

Connection Flow:

1. Host creates room → receives unique room code from signaling server
2. Host shares room code with other players (via messaging, email, etc.)
3. Other players enter room code and click Join
4. Relay server facilitates peer discovery and connection via HTTP/SSE
5. Direct peer-to-peer connections established (no server relay for game data)
6. When all players ready, host initiates game scene transition
7. GDSync framework ensures all clients load the multiplayer game scene synchronously

4.2.4 Multiplayer Game Scene

Implements the OpenMP shared-memory simulation gameplay with collaborative sorting and private buffers.

Responsibilities:

- Display shared container visible to all players (shared memory)
- Provide private buffer zones for each player (thread-local storage)
- Hide cards in other players' private buffers (thread-private data)
- Synchronize card movements across all clients
- Enable simultaneous access to shared container (parallel execution)
- Handle disconnections and reconnections gracefully
- Validate global sorting order

```
MultiplayerScene (Control)
|-- MultiplayerCardManager (Node) [Script: multiplayer_card_manager
    .gd]
|    |-- Inherits: CardManager
|    |-- GDSyncNode (Node)
|    +-- NetworkState (Dictionary)
|         |-- cards_in_other_buffers: Dictionary
|         +-- pending_sync_operations: Array
|-- UI (Control)
|    |-- TopBar (HBoxContainer)
|    |    |-- PlayerIndicator (Label) "Player 2/4"
|    |    |-- TimerLabel (Label)
|    |    +-- ConnectionStatus (TextureRect)
|    |-- CardScrollContainer (ScrollContainer)
|    |    +-- [Cards visible to this player]
|    +-- BufferContainer (VBoxContainer)
|         +-- [Private buffers for this player]
+-- ConnectionManager (Node)
     +-- [Shared with lobby, manages peer connections]
```

Listing 4.4: Multiplayer scene hierarchy

State Management: The multiplayer scene maintains several types of state:

Local State Information known only to this client (e.g., cards currently being dragged, UI state).

Private State Information owned by this player but not visible to others (e.g., cards in this player's buffers).

Shared State Information visible to all players (e.g., cards in the main container, timer, game completion status).

Authoritative State The host's version of shared state, which is the source of truth for conflict resolution.

4.3 Core Components

4.3.1 Card Component

The Card is the fundamental interactive element of the game, representing a single sortable item.

```
class_name Card
extends Control

# Card properties
var card_value: int
var card_id: String  # Unique identifier (UUID)
var current_location: String  # "container", "buffer1", "buffer2",
    etc.
var owner_player_id: String  # For multiplayer

# Visual components
@onready var panel: Panel = $Panel
@onready var label: Label = $Panel/Label
@onready var animation_player: AnimationPlayer = $AnimationPlayer

# State
var is_being_dragged: bool = false
var original_position: Vector2
var original_slot_index: int

# Signals
signal card_picked_up(card: Card)
signal card_dropped(card: Card, target_container: Control)
signal card_clicked(card: Card)
```

Listing 4.5: Card component class definition

Key Responsibilities:

- Visual representation (panel background, number label, highlighting)
- Input handling (touch/mouse drag-and-drop)
- State tracking (location, ownership, drag state)
- Animations (pickup, drop, highlighting, shake for invalid moves)
- Signal emission for game logic integration

```
func _gui_input(event: InputEvent) -> void:
    if event is InputEventScreenTouch or event is InputEventMouseButton:
        if event.pressed:
            _on_pickup(event.position)
        else:
            _on_drop(event.position)
    elif event is InputEventScreenDrag or event is InputEventMouseMotion:
        if is_being_dragged:
            _on_drag(event.position)

func _on_pickup(position: Vector2) -> void:
    is_being_dragged = true
    original_position = global_position
    z_index = 100  # Bring to front
    scale = Vector2(1.1, 1.1)  # Visual feedback
    card_picked_up.emit(self)

func _on_drop(position: Vector2) -> void:
    is_being_dragged = false
    z_index = 0
    scale = Vector2(1.0, 1.0)

    var drop_target = _get_drop_target(position)
    if drop_target:
        card_dropped.emit(self, drop_target)
    else:
        # Return to original position
        _animate_return_to_original()
```

Listing 4.6: Card drag-and-drop logic

Visual States:

- **Normal**: Default appearance
- **Highlighted**: When being dragged (scale 1.1x, slight shadow)
- **Disabled**: When in another player's buffer (semi-transparent, grayed out)
- **Invalid**: Shake animation when dropped in invalid location
- **Correct**: Green highlight when in correct sorted position

4.3.2 Card Manager

The CardManager is the central controller for game logic in single-player mode, responsible for card lifecycle, validation, and game state transitions.

```
class_name CardManager
extends Node

# Configuration
var number_of_cards: int = 50
var card_scene: PackedScene = preload("res://scenes/CardScene/Card.
    tscn")

# Game state
var cards: Array[Card] = []
var game_started: bool = false
var start_time: float = 0.0
var move_count: int = 0

# References to UI containers
@onready var card_container: HFlowContainer
@onready var buffer_zones: Array[Panel] = []

# Signals
signal game_started()
signal game_finished(time: float, moves: int)
signal card_placed_in_container(card: Card, was_in_buffer: bool,
    slot: int)
signal card_entered_buffer(card: Card, buffer_index: int)
signal card_left_buffer(card: Card, buffer_index: int)
```

Listing 4.7: CardManager class structure

```
func initialize_game() -> void:
    """Generate and shuffle cards, start timer."""
    cards.clear()
    var values = range(1, number_of_cards + 1)
    values.shuffle()

    for value in values:
        var card = card_scene.instantiate()
        card.card_value = value
        card.card_id = UUID.generate()
        card.card_dropped.connect(_on_card_dropped)
        cards.append(card)
        card_container.add_child(card)

    start_time = Time.get_ticks_msec()
    game_started = true
    game_started.emit()

func check_sorting_order() -> bool:
    """Validate if cards in container are sorted."""
    var container_cards = _get_cards_in_container()
    if container_cards.size() != cards.size():
        return false  # Some cards still in buffers

    for i in range(container_cards.size() - 1):
```

```
        if container_cards[i].card_value > container_cards[i+1].card_value:
            return false  # Not sorted

    return true

func _on_card_dropped(card: Card, target_container: Control) -> void:
    """Handle card placement in container or buffer."""
    move_count += 1

    var was_in_buffer = card.current_location.begins_with("buffer")

    if target_container == card_container:
        var slot = _calculate_drop_slot(card.global_position)
        _move_card_to_slot(card, slot)
        card.current_location = "container"

        if was_in_buffer:
            card_left_buffer.emit(card, _get_buffer_index(card))

        card_placed_in_container.emit(card, was_in_buffer, slot)

        if check_sorting_order():
            _finish_game()

    elif target_container in buffer_zones:
        var buffer_index = buffer_zones.find(target_container)
        _move_card_to_buffer(card, buffer_index)
        card.current_location = "buffer%d" % buffer_index
        card_entered_buffer.emit(card, buffer_index)

func _finish_game() -> void:
    """Handle game completion."""
    var elapsed_time = (Time.get_ticks_msec() - start_time) / 1000.0
    game_finished.emit(elapsed_time, move_count)
    _show_finish_window(elapsed_time, move_count)
```

Listing 4.8: CardManager core methods

Responsibilities Summary:

- Card generation and shuffling
- Timer management
- Move counting
- Drag-and-drop event coordination
- Buffer zone management

- Sorting validation
- Game completion detection
- Signal emission for UI updates

4.3.3 Multiplayer Card Manager

The MultiplayerCardManager extends CardManager with synchronization logic for multiplayer gameplay.

```
class_name MultiplayerCardManager
extends CardManager

# Multiplayer-specific state
var is_host: bool = false
var my_player_id: String
var connected_players: Dictionary = {}  # player_id -> player_info
var cards_in_other_buffers: Dictionary = {}  # card_id -> player_id

# GDSync node reference
@onready var gdsync: Node

# Additional signals
signal sync_required(operation: String, data: Dictionary)
signal remote_card_moved(card_id: String, new_location: String)
```

Listing 4.9: MultiplayerCardManager inheritance

Host-Authoritative Model: The multiplayer architecture follows a host-authoritative pattern where the host maintains the canonical game state and clients request state changes:

1. **Client Action**: Player drags a card and drops it
2. **Local Update**: Client immediately updates local visuals for responsiveness
3. **Broadcast to Host**: Client sends action to host via GDSync RPC
4. **Host Validation**: Host validates the move (e.g., checks if card is available)
5. **Host Broadcast**: If valid, host broadcasts confirmed state to all clients
6. **Client Reconciliation**: Clients apply authoritative update, correcting any discrepancies

```
@rpc("any_peer", "call_remote", "reliable")
func sync_card_moved(card_id: String, new_slot: int, player_id: String) -> void:
    """Synchronize card reordering in main container."""
    if not is_host:
        # Only host processes sync requests
        return

    var card = _get_card_by_id(card_id)
    if card:
        _move_card_to_slot(card, new_slot)
        # Broadcast to all clients
        rpc("apply_card_moved", card_id, new_slot)

@rpc("authority", "call_remote", "reliable")
func apply_card_moved(card_id: String, new_slot: int) -> void:
    """Apply card movement from authoritative host."""
    var card = _get_card_by_id(card_id)
    if card:
        _move_card_to_slot(card, new_slot)

@rpc("any_peer", "call_remote", "reliable")
func sync_card_entered_buffer(card_id: String, player_id: String, buffer_index: int) -> void:
    """Synchronize card moving to a player's private buffer."""
    if not is_host:
        return

    # Record which player has this card
    cards_in_other_buffers[card_id] = player_id

    # Broadcast to all clients to hide the card
    rpc("apply_card_entered_buffer", card_id, player_id, buffer_index)

@rpc("authority", "call_remote", "reliable")
func apply_card_entered_buffer(card_id: String, player_id: String, buffer_index: int) -> void:
    """Hide card that entered another player's buffer."""
    if player_id == my_player_id:
        return  # Owner still sees their own cards

    var card = _get_card_by_id(card_id)
    if card:
        card.visible = false
        card.set_process_input(false)  # Disable interaction

@rpc("any_peer", "call_remote", "reliable")
func sync_card_left_buffer(card_id: String, player_id: String) -> void:
    """Synchronize card leaving a player's buffer back to container."""
    if not is_host:
        return

    if card_id in cards_in_other_buffers:
```

```
        cards_in_other_buffers.erase(card_id)

    # Broadcast to make card visible again
    rpc("apply_card_left_buffer", card_id)

@rpc("authority", "call_remote", "reliable")
func apply_card_left_buffer(card_id: String) -> void:
    """Show card that returned from a buffer."""
    var card = _get_card_by_id(card_id)
    if card:
        card.visible = true
        card.set_process_input(true)

func sync_complete_game_state(target_peer_id: int) -> void:
    """Send full game state to newly connected player."""
    if not is_host:
        return

    var state = {
        "cards": _serialize_card_state(),
        "timer": get_elapsed_time(),
        "moves": move_count,
        "buffers": cards_in_other_buffers.duplicate()
    }

    rpc_id(target_peer_id, "apply_complete_game_state", state)
```

Listing 4.10: Key synchronization methods

Visibility Management: A critical aspect of the OpenMP simulation is maintaining correct visibility of cards to represent thread-private data:

- **Cards in shared container**: Visible to all players (shared memory)
- **Cards in my private buffers**: Visible only to me (my thread-local storage)
- **Cards in other players' private buffers**: Hidden from view (other threads' private data)
- **Cards being moved**: Brief transition animation showing memory access

The `cards_in_other_buffers` dictionary tracks which cards are currently private:

```
var cards_in_other_buffers: Dictionary = {}
# Example state:
# {
#   "card_uuid_123": "player_2",
#   "card_uuid_456": "player_2",
#   "card_uuid_789": "player_3"
# }
```

Listing 4.11: Visibility tracking

4.3.4 Scroll Container and Layout

Managing 50–200 cards on screen requires efficient layout and scrolling.

```
CardScrollContainer (ScrollContainer)
|-- scroll_vertical_enabled = true
|-- scroll_horizontal_enabled = false
+-- CardFlowContainer (HFlowContainer)
    |-- alignment = ALIGNMENT_BEGIN
    |-- separation = 10
    +-- [Card instances]
```

Listing 4.12: Scrollable card container structure

Layout Strategy:

- **HFlowContainer**: Automatically wraps cards to multiple rows based on available width
- **Responsive Card Size**: Cards scale based on screen width to fit approximately 4–6 per row
- **Vertical Scrolling Only**: Horizontal scrolling disabled for simpler interaction
- **Input-Optimized**: Large click/touch targets, smooth scroll physics

```
func _calculate_drop_slot(drop_position: Vector2) -> int:
    """Determine which slot a card should be dropped into."""
    var container_cards = card_container.get_children()

    # Convert global position to local container position
    var local_pos = card_container.to_local(drop_position)

    # Iterate through existing cards to find insertion point
    for i in range(container_cards.size()):
        var card = container_cards[i]
        var card_center = card.position + card.size / 2

        if local_pos.x < card_center.x:
            return i

    # Drop after all cards
    return container_cards.size()
```

Listing 4.13: Drop slot calculation

4.4 Multiplayer Architecture

Drop Target Detection:

4.4.1 Network Topology

The game uses a **peer-to-peer (P2P) mesh network** topology with **host-authoritative state management**.

Topology Diagram:

```
Player 1 (Host)
    |  \  \
    |   \  \___
    |    \      \
Player 2  Player 3  Player 4
    |   ___/     /
    |  /    ____/
    | /    /
   \|/   |/
Direct P2P Connections
```

Connection Characteristics:

- Each player connects through the relay server to communicate with other players
- No dedicated server required for gameplay (signaling server only used for initial connection)
- Low latency for small groups (2–4 players)
- Host election occurs if original host disconnects

4.4.2 State Synchronization Patterns

Pattern 1: Optimistic Update with Confirmation Used for non-critical actions where responsiveness is important:

1. Client performs action locally (e.g., moving card within own buffer)
2. Client updates local visuals immediately
3. Client sends action to host

4. Host validates and broadcasts
5. If host rejects, client receives correction and reverts

Pattern 2: Host Broadcast Used for host-initiated events:

1. Host detects event (e.g., game timer expires, sorting complete)
2. Host broadcasts event to all clients
3. Clients apply event simultaneously

4.4.3 GDSync Integration

GDSync is a third-party multiplayer framework for Godot that provides high-level abstractions for state synchronization. For web browser compatibility, **a custom HTTP/SSE transport layer was implemented** to replace GDSync's native networking. The game primarily interacts with GDSync's RPC API, which then uses the custom transport layer for browser-based communication.

Key Features Used:

- **RPC (Remote Procedure Calls)**: Call functions on remote peers
- **Node Replication**: Automatically sync node properties
- **Ownership System**: Track which peer owns which nodes
- **Authority Modes**: Specify who can call functions ("any_peer", "authority")
- **Transport Abstraction**: Custom HTTP/SSE transport layer provides browser-compatible networking

```
func _ready():
    super._ready()  # Call CardManager._ready()

    # Initialize GDSync
    gdsync = $GDSyncNode
    gdsync.set_multiplayer_authority(1)  # Host = peer ID 1

    # Register synchronized functions
    gdsync.register_rpc("sync_card_moved",
                         Callable(self, "sync_card_moved"))
    gdsync.register_rpc("sync_card_entered_buffer",
                         Callable(self, "sync_card_entered_buffer"))
    gdsync.register_rpc("sync_card_left_buffer",
                         Callable(self, "sync_card_left_buffer"))
```

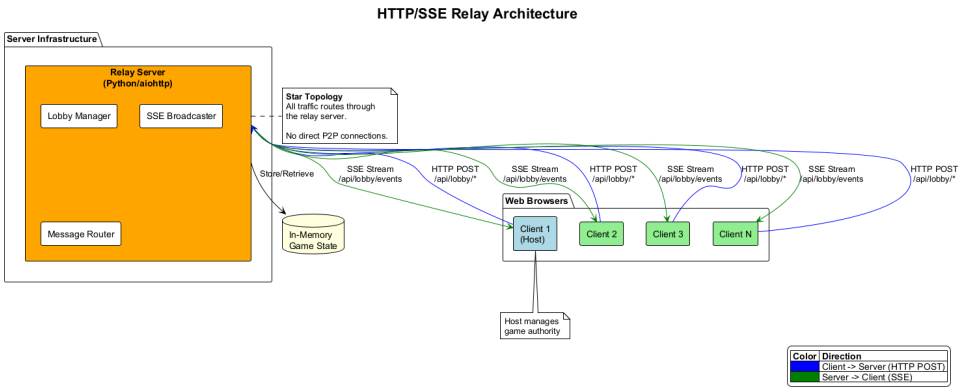


Figure 4.4: HTTP/SSE relay architecture: star topology with centralized relay server

```
    # Connect to networking events
    multiplayer.peer_connected.connect(_on_peer_connected)
    multiplayer.peer_disconnected.connect(_on_peer_disconnected)
```

Listing 4.14: GDSync setup in MultiplayerCardManager

```
# Pattern 1: Broadcast to all peers
func broadcast_card_moved(card_id: String, slot: int):
    rpc("sync_card_moved", card_id, slot, my_player_id)

# Pattern 2: Call specific peer
func send_to_host(data: Dictionary):
    rpc_id(1, "process_client_action", data)

# Pattern 3: Host broadcasts authoritative update
func host_broadcast_state():
    if is_host:
        rpc("apply_card_moved", card_id, new_slot)
```

Listing 4.15: Using GDSync RPCs

4.4.4 Connection Management

RPC Call Patterns:

Lobby Phase:

1. Host creates room → signaling server assigns unique code
2. Host shares code with players

3. Players connect via relay server
4. Each player establishes direct P2P connection with all others
5. Host waits for all players ready
6. Host triggers synchronized scene transition

Gameplay Phase:

1. Host initializes game state (shuffle cards, start timer)
2. Host broadcasts initial state to all clients
3. Clients apply initial state and begin gameplay
4. Players send actions via RPC
5. Host validates and broadcasts confirmations
6. Game progresses until completion or disconnection

```
func _on_peer_disconnected(peer_id: int):
    var player_name = connected_players[peer_id].name
    ToastParty.show_toast("%s disconnected" % player_name)

    connected_players.erase(peer_id)

    # If host disconnected, elect new host
    if peer_id == 1 and not is_host:
        _elect_new_host()

    # Release cards owned by disconnected player
    for card_id in cards_in_other_buffers.keys():
        if cards_in_other_buffers[card_id] == peer_id:
            cards_in_other_buffers.erase(card_id)
            _return_card_to_container(card_id)

func _elect_new_host():
    """Simple host election: lowest remaining peer ID becomes host.
    """
    var peer_ids = connected_players.keys()
    peer_ids.sort()
    var new_host_id = peer_ids[0]

    if multiplayer.get_unique_id() == new_host_id:
        is_host = true
        ToastParty.show_toast("You are now the host")
        gdsync.set_multiplayer_authority(new_host_id)
```

Listing 4.16: Handling player disconnections

4.5 Data Flow Diagrams

Disconnection Handling:

4.5.1 Single-Player Game Flow

```
User Input (Touch)
    |
    v
Card._gui_input()
    |
    v
Card.card_dropped.emit(card, target_container)
    |
    v
ScrollContainer._on_card_dropped() [signal forwarding]
    |
    v
CardManager._on_card_placed_in_container(card, was_in_buffer, slot)
    |
    +---> Update card position in container
    +---> Increment move counter
    +---> If was_in_buffer: emit card_left_buffer signal
    +---> emit card_placed_in_container signal
    |
    v
CardManager.check_sorting_order()
    |
    +---> If not sorted: continue gameplay
    |
    +---> If sorted:
          |
          v
          CardManager._finish_game()
              |
              +---> Stop timer
              +---> Calculate elapsed time
              +---> emit game_finished signal
              +---> Show finish window with confetti
```

4.5.2 Multiplayer Card Movement Flow

Scenario: Player 2 moves a card in the shared container

```
Player 2 Client:
```

```
    User drags card to new position
        |
        v
    Card.card_dropped.emit()
        |
        v
    MultiplayerCardManager._on_card_placed_in_container()
        |
        +---> Update local visuals (immediate feedback)
        +---> Calculate new slot index
        |
        v
    RPC: sync_card_moved(card_id, new_slot, "player_2")
        |
        | [Network transmission]
        v

Host (Player 1):
    Receives sync_card_moved RPC
        |
        v
    Validate move (check card availability)
        |
        +---> If invalid: send rejection (not implemented yet)
        |
        +---> If valid:
              |
              v
              Apply move to host's game state
              |
              v
              RPC broadcast: apply_card_moved(card_id, new_slot)
                  |
                  | [Broadcast to all clients]
                  v

All Clients (including Player 2):
    Receive apply_card_moved RPC
        |
        v
    Apply authoritative card position
        |
        v
    Update UI: _move_card_to_slot(card, new_slot)
        |
        v
```

```
    Animation: smooth transition to new position
```

4.5.3 Buffer Visibility Synchronization

Scenario: Player 3 takes a card into their private buffer

```
Player 3 Client:
    User drags card from container to buffer zone
        |
        v
    Card.card_dropped.emit(card, buffer_panel)
        |
        v
    MultiplayerCardManager._on_card_dropped()
        |
        +---> Determine target is buffer
        +---> Move card to buffer visually
        +---> Update card.current_location = "buffer1"
        |
        v
    RPC: sync_card_entered_buffer(card_id, "player_3", buffer_index)
        |
        | [Network transmission]
        v

Host (Player 1):
    Receives sync_card_entered_buffer RPC
        |
        v
    Record: cards_in_other_buffers[card_id] = "player_3"
        |
        v
    RPC broadcast: apply_card_entered_buffer(card_id, "player_3", buffer_index)
        |
        | [Broadcast to all clients]
        v

Player 1 & Player 2 Clients (others):
    Receive apply_card_entered_buffer RPC
        |
        v
    Check: Is this my card? (player_id == my_player_id)
        |
        +---> Yes (should not happen): do nothing
        |
        +---> No (card belongs to Player 3):
```

```
                |
                v
                Hide card: card.visible = false
                Disable interaction: card.set_process_input(false)

Player 3 Client (owner):
    Receive apply_card_entered_buffer RPC
        |
        v
    Check: Is this my card? YES
        |
        v
    Do nothing (card remains visible and interactive)
```

Result:

- Player 3 sees the card in their buffer and can interact with it
- Players 1, 2, and 4 do not see the card (simulating distributed memory)
- Card remains in game state but is hidden from other players' views
- When Player 3 returns the card to the container, `sync_card_left_buffer` makes it visible again to everyone

4.6 Responsive UI/UX Design

4.6.1 Design Constraints

Web platforms and varying screen sizes impose several constraints that significantly influenced design decisions:

- **Screen Size**: 5–7 inch displays, approximately 1080x2400 pixels
- **Touch Input**: Fingers occlude content, require larger targets (minimum 44x44 dp)
- **Performance**: Variable GPU/CPU capabilities, battery constraints
- **Network**: Variable bandwidth, potential disconnections
- **Orientation**: Support both portrait and landscape

4.6.2 Layout Strategies

Card Display:

- Cards sized dynamically: `min(screen_width / 6, 120px)`
- Approximately 4–6 cards per row on typical phones
- 10–20 rows visible depending on card count and screen size
- Vertical scrolling for navigation

Buffer Zones:

- Fixed height panels at bottom of screen
- 2–3 buffers per player (configurable)
- Each buffer displays up to 10–15 cards in a horizontal flow
- Collapse/expand toggles to save screen space

Top Bar:

- Fixed position header with timer, move count, player indicator
- Always visible during gameplay
- Minimal height to maximize card display area

4.6.3 Touch Interaction Patterns

Drag-and-Drop:

- Long-press to pick up card (500ms threshold prevents accidental drags)
- Visual feedback: card scales to 1.1x, drops shadow, follows finger
- Drag anywhere on screen
- Drop zones highlighted when card is near
- Snap-to-slot animation when dropped
- Shake animation if dropped in invalid location

Scrolling vs. Dragging:

- ScrollContainer consumes vertical swipes
- Card drag consumes touch once long-press threshold reached
- Clear visual distinction: cards grow when draggable

4.6.4 Visual Feedback Systems

Toast Notifications: Brief non-intrusive messages for events:

- "Card entered buffer"
- "Player 3 joined"
- "Player disconnected"
- "Game finished!"

Animations:

- **Card Pickup**: Scale up, add shadow (200ms)
- **Card Drop**: Scale down, fade shadow (200ms)
- **Confetti**: Particle effect on game completion (3000ms)

Color Coding:

- **Normal Cards**: Light blue background, black text
- **Highlighted Cards**: Yellow background (when dragging)
- **Hidden Cards**: 30% opacity, gray (in other players' buffers)
- **Buffers**: Slightly darker background to distinguish from main container

4.7 Performance Considerations

4.7.1 Optimization Strategies

Rendering:

- Cards use simple Panel nodes (not full Control trees) to minimize draw calls

- Visibility culling: cards outside scroll view not rendered
- Animations use Tween for GPU-accelerated interpolation
- Limit particle effects (confetti) to 100–200 particles

Networking:

- Aggregate multiple card movements before sending (debounce 100ms)
- Use "reliable" RPC only for critical state changes
- Use "unreliable" for non-critical updates (position interpolation)
- Compress state synchronization messages

Memory:

- Reuse card instances instead of destroying/recreating
- Preload frequently used resources (card scene, textures)
- Clear large data structures when switching scenes
- Limit maximum cards to 200 to prevent memory issues

4.7.2 Scalability Limits

Based on testing and design constraints:

- **Maximum Cards**: 200 (UI becomes difficult with more)
- **Maximum Players**: 4 (P2P mesh becomes inefficient beyond this)
- **Target Frame Rate**: 60 FPS in modern web browsers
- **Network Latency Tolerance**: Acceptable gameplay up to 200ms RTT
- **Minimum Screen Size**: 5-inch display (smaller makes cards too small)

4.8 Summary

This chapter presented the comprehensive architecture of the HPC Sorting Serious Game, covering:

- High-level system structure and design principles

- Scene organization and responsibilities
- Core component designs (Card, CardManager, MultiplayerCardManager)
- Multiplayer networking architecture with host-authoritative model
- GDSync framework integration for state synchronization
- Detailed data flow diagrams for single-player and multiplayer scenarios
- Responsive UI/UX design strategies and constraints
- Performance optimization considerations

The architecture balances educational effectiveness with technical feasibility, prioritizing clarity of HPC concept representation while maintaining acceptable performance across web browsers. The modular, component-based design allows for future extensions and modifications, fulfilling the extensibility objective outlined in Chapter 1.

The next chapter will detail the implementation of this architecture, including development tools, code organization, and specific technical challenges encountered during realization of this design.

Chapter 5

Implementation

This chapter describes the detailed technical implementation of the HPC Sorting Serious Game, covering both single-player and multiplayer modes, key algorithms, code structure, and mobile-specific optimizations.

5.1 Development Environment Setup

5.1.1 Project Structure

The project follows Godot's standard directory structure with additional organization for plugins and resources:

```
hpc-sorting-serious-game/
|-- scenes/                # Game scenes
|    |-- main_menu.tscn
|    |-- lobby.tscn
|    |-- singleplayer.tscn
|    |-- multiplayer.tscn
|-- scripts/               # GDScript source files
|    |-- card.gd
|    |-- card_manager.gd
|    |-- buffer.gd
|    |-- lobby_manager.gd
|-- resources/             # Assets and resources
|    |-- textures/
|    |-- fonts/
|    |-- audio/
|-- addons/                # Third-party plugins
|    |-- GD-Sync/
|    |-- ToastParty/
|    |-- Logger/
|-- project.godot          # Godot project configuration
```

Listing 5.1: Project directory structure

5.2 Core Components Implementation

5.2.1 Card Component

The Card component represents an individual playing card and handles rendering, interaction, and state management.

Key Responsibilities:

- Display card number and visual state
- Handle drag-and-drop interactions
- Manage selection and highlighting
- Emit signals for game logic coordination

```
extends Control
class_name Card

signal card_selected(card: Card)
signal card_dragged(card: Card, position: Vector2)
signal card_dropped(card: Card, target)

var card_value: int
var original_position: Vector2
var is_dragging: bool = false
var is_selected: bool = false

func _ready():
    # Initialize card visuals
    update_display()

func update_display():
    # Update label with card value
    $Label.text = str(card_value)
    # Update visual state based on flags
    if is_selected:
        modulate = Color.YELLOW
    else:
        modulate = Color.WHITE
```

Listing 5.2: Card component structure (card.gd)

5.2.2 Card Manager

Implementation Details: The Card Manager is responsible for creating, shuffling, and managing the collection of cards.

Key Responsibilities:

- Generate card instances
- Shuffle and distribute cards
- Validate sorting correctness
- Manage card ownership in multiplayer

5.2.3 Buffer System

Buffers represent thread-local storage in OpenMP shared-memory parallelism.

Implementation:

- Each player has 2-3 buffer zones
- Cards can be dragged into buffers
- Buffers maintain sorted or unsorted card lists
- Visibility controlled based on game mode

5.3 Single-Player Mode Implementation

5.3.1 Game Initialization

Steps for initializing a single-player game:

1. Load singleplayer scene
2. Generate specified number of cards
3. Shuffle cards randomly
4. Place cards in main container
5. Initialize buffers
6. Start timer

5.3.2 Drag-and-Drop System

The drag-and-drop system is crucial for mobile touch interaction.

Touch Event Handling:

- `_input(event)`: Detects touch/mouse events
- `_process(delta)`: Updates card position during drag
- Drop zones detect overlap with dragged cards

5.3.3 Sorting Validation

Algorithm to check if cards are sorted:

```
func is_sorted(cards: Array) -> bool:
    if cards.size() < 2:
        return true

    for i in range(cards.size() - 1):
        if cards[i].card_value > cards[i + 1].card_value:
            return false

    return true
```

Listing 5.3: Sorting validation algorithm

5.4 Multiplayer Mode Implementation

5.4.1 Lobby System

The lobby manages room creation, player joining, and game configuration.

Lobby Flow:

1. Player creates room or enters room code
2. Connection to signaling server
3. WebRTC handshake with host
4. Players appear in lobby list
5. Host configures game settings
6. Host starts game

5.4.2 GDSync Integration

GDSync provides high-level networking abstractions.

Key Features Used:

- `@GDSync.sync`: Mark nodes for automatic synchronization
- `@GDSync.rpc`: Remote procedure calls
- Connection management
- Lobby synchronization

```
# Mark card container for synchronization
@GDSync.sync(mode=GDSync.SYNC_MODE.AUTHORITY)
var card_container: Node

# Remote procedure call for card movement
@GDSync.rpc(call_local=true)
func move_card_networked(card_id: int, target_container_id: int):
    # Move card on all clients
    var card = get_card_by_id(card_id)
    var target = get_container_by_id(target_container_id)
    target.add_child(card)
```

Listing 5.4: GDSync node configuration

5.4.3 State Synchronization

Configuration: Maintaining consistent game state across clients is critical.

Synchronization Strategy:

- **Host-Authoritative**: Host has authority over game state
- **Client Prediction**: Clients show immediate local updates
- **Server Reconciliation**: Host sends authoritative updates
- **Delta Updates**: Only changed data transmitted

Synchronized Data:

- Card positions and container ownership
- Player states (connected, ready, playing)
- Timer and move counter
- Game phase (lobby, playing, finished)

5.4.4 Visibility Management

In multiplayer mode (OpenMP simulation), players should not see cards in other players' private buffers, representing thread-private data.

```
func update_card_visibility():
    for card in all_cards:
        if card.is_in_private_buffer():
            var buffer_owner_id = card.get_buffer_owner()
            if buffer_owner_id == local_player_id:
                card.visible = true
            else:
                card.visible = false  # Hide other players' cards
        else:
            card.visible = true  # Shared container cards visible
```

Listing 5.5: Card visibility control

5.5 Mobile Optimizations

Implementation:

5.5.1 Performance Optimizations

Rendering Optimizations:

- Object pooling for cards (reuse instances)
- Viewport culling (don't render off-screen cards)
- Batch rendering where possible
- Minimize draw calls

Memory Management:

- Preload commonly used resources
- Release unused assets
- Texture compression for mobile
- Minimize garbage collection pauses

5.5.2 Touch Input Optimization

Touch Target Sizing:

- Minimum 48x48 dp for interactive elements
- Adequate spacing between cards
- Visual feedback for touch events

5.5.3 Network Optimization

Data Minimization:

- Send only changed data
- Compress network messages
- Batch multiple updates when possible
- Predictive client-side updates

5.6 Debugging and Development Tools

5.6.1 Logger Plugin Integration

The Logger plugin provides categorized logging for debugging.

```
Logger.info("Card %d moved to buffer %s" % [card.card_value, buffer
    .name])
Logger.warn("Network latency high: %d ms" % latency)
Logger.error("Failed to synchronize card state")
```

5.6.2 Runtime Variable Inspection

Usage Example: VarTree plugin allows real-time inspection and modification of variables during gameplay for debugging purposes. This tool displays GDSync connection state variables such as client ID, host status, current lobby, and player count.

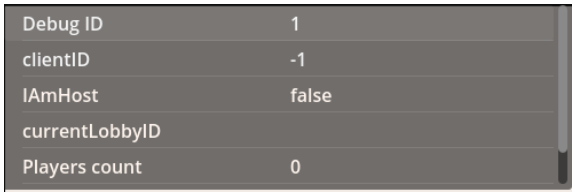


Figure 5.1: VarTree debug panel showing GDSync runtime variables: Debug ID, clientID, IAmHost flag, currentLobbyID, and Players count

5.7 Code Quality and Best Practices

5.7.1 Code Organization

Principles Followed:

- Single Responsibility Principle
- Separation of concerns (UI, logic, networking)
- DRY (Don't Repeat Yourself)
- Clear naming conventions

5.7.2 Documentation

All scripts include:

- File-level documentation describing purpose
- Function documentation with parameters and return values
- Inline comments for complex logic
- Signal documentation

5.8 Summary

This chapter presented the detailed implementation of the HPC Sorting Serious Game, covering:

- **Project Structure**: Organization of scenes, scripts, and resources
- **Core Components**: Card, Card Manager, and Buffer implementations
- **Single-Player Mode**: Initialization, drag-and-drop, and sorting validation

- **Multiplayer Mode**: Lobby system, GDSync integration, state synchronization, and visibility management
- **Mobile Optimizations**: Performance, touch input, and network optimizations
- **Development Tools**: Debugging and logging utilities
- **Code Quality**: Organization and documentation practices

The next chapter discusses the problems and challenges encountered during development and the solutions implemented to address them.

Chapter 6

Problems and Challenges

This chapter provides an honest and comprehensive analysis of the difficulties encountered during the development of the HPC Sorting Serious Game. Understanding these challenges and their solutions is valuable for future developers of educational multiplayer games and contributes to the body of knowledge in serious game development.

6.1 Overview

Development of a serious game involving real-time multiplayer synchronization on mobile devices presented numerous technical, design, and pedagogical challenges. These problems span multiple domains:

- Technology selection and framework limitations
- Multiplayer state synchronization complexity
- Mobile UI/UX constraints
- Performance optimization requirements
- Documentation and community support gaps
- Testing and debugging difficulties

This chapter documents these challenges, the approaches taken to address them, and lessons learned that may benefit future projects.

6.2 Technology Selection Challenges

6.2.1 Game Engine Trade-offs

Challenge: While Godot Engine proved to be a good choice overall, several limitations became apparent during development:

- **Smaller Ecosystem**: Fewer third-party plugins and assets compared to Unity, requiring custom solutions for some features
- **Documentation Gaps**: Godot 4.x was relatively new, with incomplete documentation for advanced networking scenarios
- **Mobile Export Issues**: Occasional problems with Android export templates and permission configurations
- **Plugin Compatibility**: Some Godot 3.x plugins not yet updated for 4.x

Impact: Development time increased due to the need to:

- Build custom solutions rather than using existing plugins
- Troubleshoot export issues through trial and error
- Consult community forums for undocumented features

Mitigation:

- Active participation in Godot community (Discord, forums, GitHub)
- Contribution to documentation through issue reports and examples
- Development of reusable utility scripts for common tasks
- Careful version management and testing before engine updates

6.2.2 GDScript Performance Concerns

Challenge: GDScript's interpreted nature raised concerns about performance with 50–200 cards requiring frequent position updates and collision detection.

Solution: Profiling revealed that:

- GDScript performance was acceptable for game logic
- Bottlenecks were in rendering and physics, handled by Godot's C++ core
- Network latency dominated perceived lag, not scripting performance

Optimization strategies:

- Minimize expensive operations in `_process()` and `_physics_process()`
- Use object pooling to reduce instantiation overhead
- Batch updates where possible
- Profile early and often to identify actual bottlenecks

6.3 GDSync Framework Challenges

The most significant technical challenges involved the GDSync framework, which, despite its high-level abstractions, presented several issues.

6.3.1 Protected Mode Blocking Issue

Challenge:

GDSync's "protected mode" (default) prevented RPC calls from being executed between non-host peers. When Player A tried to call an RPC on Player B's node, the call would be silently dropped or blocked.

Error Manifestation:

- Card movement worked from host to clients but not between clients
- No error messages or warnings—calls simply failed silently
- Debugging difficult due to lack of visibility into framework internals

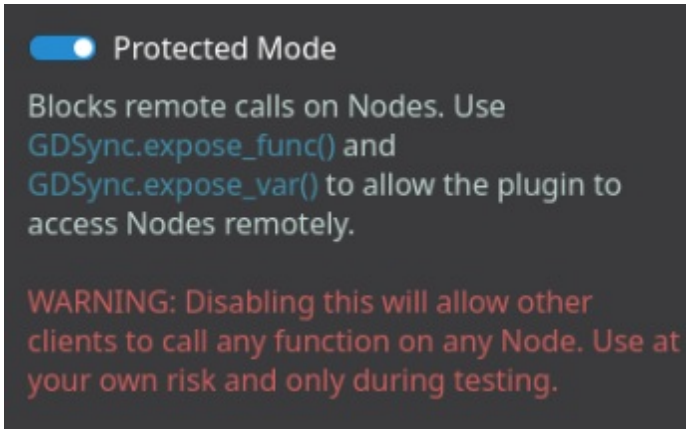


Figure 6.1: GDSync's Protected Mode toggle in the plugin settings. When enabled (default), it blocks remote calls on Nodes that haven't been explicitly exposed via `GDSync.expose_func()` or `GDSync.expose_var()`

Investigation Process:

1. Verified RPC syntax was correct according to documentation
2. Added extensive logging to track RPC call flow
3. Examined GDSync source code to understand permission system

4. Tested with GDSync's example projects to compare behavior
5. Posted issue on GDSync GitHub repository

Solution:

- Disabled "protected mode" in GDSync configuration
- Implemented custom permission checks in application logic
- Contributed documentation improvements to GDSync project
- Shared findings with community to help future users

Lesson Learned: When using third-party frameworks:

- Read source code, not just documentation
- Test framework features in isolation before integration
- Maintain good logging and diagnostic capabilities
- Engage with framework maintainers early when issues arise

6.3.2 Incomplete Documentation

Challenge:

GDSync documentation covered basic use cases but lacked:

- Examples of complex synchronization scenarios
- Explanation of internal mechanisms and limitations
- Best practices for performance optimization
- Troubleshooting guides for common problems
- Migration guide from vanilla Godot networking

Impact:

- Trial-and-error approach required for advanced features
- Difficulty distinguishing between usage errors and framework bugs
- Increased development time
- Frustration and consideration of framework replacement

Mitigation:

- Created internal documentation of discoveries and workarounds
- Maintained test project for isolating framework behavior
- Contributed examples and documentation improvements to upstream project
- Built abstraction layer to isolate GDSync-specific code

6.3.3 Debugging Multiplayer Issues

Challenge:

Debugging multiplayer code is inherently difficult:

- Requires multiple devices or instances
- Race conditions and timing-dependent bugs
- Network variability complicates reproduction
- Limited visibility into remote client state

Solutions Implemented:

1. **Multi-Instance Testing**: Launched multiple Godot editor instances on the same machine
2. **Network Logging**: Comprehensive logging of all network events with timestamps
3. **State Dumps**: Ability to print full game state on command
4. **Visual Indicators**: On-screen display of connection status, peer IDs, sync status
5. **Replay System**: Recorded game events for post-mortem analysis

6.4 Multiplayer Synchronization Challenges

6.4.1 Card Order Synchronization

Challenge:

Maintaining consistent card ordering across clients proved more complex than anticipated. Issues included:

- Godot's scene tree ordering not guaranteed to match logical ordering
- Child node order not automatically synchronized
- Timing differences causing transient inconsistencies

Symptom:

Cards appeared in different orders on different clients' screens, even though the underlying data was correct.

Solution:

1. Implemented explicit position indices for cards
2. Synchronized card order separately from card positions
3. Used z-index to control visual stacking order
4. Periodic consistency checks and resynchronization

```
# Each card has an explicit order index
var card_order_index: int

# Synchronize order explicitly
@GDSync.rpc(call_local=true)
func sync_card_order(card_id: int, new_index: int):
    var card = get_card_by_id(card_id)
    card.card_order_index = new_index
    resort_cards_by_index()
```

Listing 6.1: Card order synchronization approach

6.4.2 Timing and Race Conditions

Challenge:

Multiplayer systems are susceptible to race conditions:

- Simultaneous card movements by different players
- Network messages arriving out of order
- Inconsistent state when players join mid-game

Examples:

- Two players drag the same card simultaneously
- Player joins lobby after game has started

- Card deleted on host but still referenced on client

Solutions:

- **Host Authority**: Only host can finalize state changes
- **Client Prediction with Rollback**: Show immediate feedback, then correct if host disagrees
- **State Snapshots**: New players receive full state snapshot on join
- **Defensive Programming**: Check object existence before operations

6.4.3 Different Client Views

Challenge:

In multiplayer mode (OpenMP simulation), different players should see different cards in private buffers (simulating thread-private data), but maintaining this selective visibility while synchronizing the shared container was complex.

Technical Issue: GDSync's node replication tries to keep all clients' scene trees identical, but selective visibility required different scene trees per client.

Solution:

- Replicate all cards to all clients (for consistency)
- Use visibility flags to hide inappropriate cards
- Implement client-side filtering based on ownership metadata
- Synchronize ownership and visibility separately from positions

6.5 Mobile UI/UX Challenges

6.5.1 Displaying Many Cards on Small Screens

Challenge:

Displaying 50–200 cards on a 5–6 inch mobile screen presents significant layout challenges:

- Cards must be large enough to read and tap
- All cards must be accessible without excessive scrolling
- Layout must work in both portrait and landscape orientations

Solutions Attempted:

1. **Grid Layout**: Cards arranged in a grid
   - Pros: Space-efficient, familiar pattern
   - Cons: Hard to see linear order, scrolling still required
2. **Scrollable Row**: Single row with horizontal scrolling
   - Pros: Linear order visible, simple interaction
   - Cons: Requires substantial scrolling with many cards
3. **Zoomable Canvas**: Pan and zoom to navigate cards
   - Pros: Flexible navigation, can fit many cards
   - Cons: Complex interaction, easy to get lost
4. **Hybrid Approach (Final Solution)**:
   - Grid layout with intelligent sizing
   - Vertical scrolling for overflow
   - Zoom controls for adjusting card size
   - Mini-map showing overall progress

6.5.2 Touch Interaction Precision

Challenge:

Touch input is less precise than mouse input:

- Finger occludes card being dragged
- Difficulty selecting specific cards when densely packed
- Accidental touches and swipes

Solutions:

- **Offset Dragging**: Card rendered above finger with offset
- **Magnification**: Enlarge selected card for better visibility
- **Touch Feedback**: Visual and haptic feedback for touch events
- **Double-Tap Select**: Alternative to drag for card selection
- **Drop Zone Highlighting**: Clear indication of valid drop targets

6.5.3 Performance on Low-End Devices

Challenge:

Not all students have flagship smartphones. The game needed to run acceptably on low-end and mid-range devices from 2018–2020.

Performance Constraints:

- Limited GPU power for rendering many cards
- Lower memory (2–4 GB RAM)
- Slower network hardware
- Battery life concerns

Optimization Strategies:

- Reduce draw calls through batching
- Use texture atlases to minimize state changes
- Implement level-of-detail (LOD) for distant cards
- Viewport culling to avoid rendering off-screen cards
- Adjustable quality settings (card detail, effects)

6.6 Testing Challenges

6.6.1 Multiplayer Testing Complexity

Challenge:

Testing multiplayer functionality requires:

- Multiple devices or instances
- Coordination between test clients
- Reproduction of specific network conditions
- Testing edge cases (disconnections, late joins, etc.)

Approaches:

1. **Multi-Instance Desktop Testing**: Run multiple Godot instances on development machine
2. **Physical Device Testing**: Test on actual Android phones
3. **Automated Test Scenarios**: Scripts to simulate player actions
4. **Network Simulation**: Tools to introduce latency and packet loss

Limitations:

- Automated testing limited for interactive gameplay
- Difficult to reproduce exact timing of race conditions
- Real-world network conditions vary widely

6.6.2 Lack of Automated Testing

Challenge:

Godot's testing ecosystem is less mature than traditional software development frameworks:

- No built-in unit testing framework
- GUI testing particularly difficult
- Networking code hard to test in isolation

Approach Taken:

- Manual testing with documented test cases
- Integration tests for critical paths
- Regression testing checklist before releases
- Community feedback as informal usability testing

Future Improvement:

- Integrate GUT (Godot Unit Test) framework
- Develop automated test suite for core logic
- Implement continuous integration pipeline

6.7 Educational Design Challenges

6.7.1 Balancing Accuracy and Playability

Challenge:

Educational games must balance pedagogical accuracy with engaging gameplay. Too much realism can reduce fun; too much simplification can create misconceptions.

Trade-offs Made:

- **Simplified Memory Model**: Actual shared-memory systems have complex cache hierarchies not represented
- **Abstracted Synchronization**: Real OpenMP requires explicit barriers, locks, and atomic operations
- **Idealized Performance**: Network latency doesn't perfectly model memory access patterns

Mitigation:

- Clear framing: "This is a metaphor, not a simulation"
- Post-game discussions to connect game to real concepts
- Accompanying documentation explaining simplifications
- Progressive difficulty introducing more complexity

6.7.2 Assessment of Learning Effectiveness

Challenge:

Demonstrating that the game actually improves learning outcomes requires formal educational research, including:

- Pre/post-test measurements
- Control groups using traditional instruction
- Statistical analysis of results
- Long-term retention studies

Limitations: Within the scope of this thesis, full educational efficacy studies were not feasible due to:

- Time constraints
- Need for institutional review board approval
- Difficulty recruiting sufficient participants
- Complexity of isolating game's effect from other factors

Future Work: Formal educational evaluation remains an important direction for future research, as discussed in Chapter 8.

6.8 Lessons Learned

6.8.1 Technical Lessons

1. **Framework Evaluation**: Thoroughly evaluate third-party frameworks before committing; test advanced use cases early.
2. **Abstraction Layers**: Isolate framework-specific code to facilitate future replacement if needed.
3. **State Management**: Design clear state management patterns early; retrofitting is expensive.
4. **Logging Infrastructure**: Comprehensive logging is essential for multiplayer debugging.
5. **Performance Profiling**: Profile early and often; don't assume where bottlenecks are.

6.8.2 Design Lessons

1. **User Testing**: Conduct user testing early; designer assumptions often wrong.
2. **Mobile-First**: Design for mobile constraints from the start, not as an afterthought.
3. **Simplicity**: Simpler interactions are better on mobile; complexity reduces usability.
4. **Feedback**: Clear, immediate feedback is critical for learning and engagement.

6.8.3 Process Lessons

1. **Iterative Development**: Iterative, incremental development surfaced issues early.
2. **Version Control**: Frequent commits with clear messages saved time during debugging.
3. **Documentation**: Documenting problems and solutions as they occurred proved invaluable.
4. **Community Engagement**: Engaging with open-source communities provided solutions and support.

6.9 Summary

This chapter documented the significant challenges encountered during development:

- **Technology Challenges**: Engine trade-offs, framework limitations, and performance concerns
- **GDSync Issues**: Protected mode blocking, incomplete documentation, and debugging difficulty
- **Synchronization Complexity**: Card ordering, race conditions, and selective visibility
- **Mobile UI/UX**: Screen space constraints, touch precision, and device performance variability
- **Testing Difficulty**: Multiplayer testing complexity and lack of automated testing tools
- **Educational Design**: Balancing accuracy with playability and assessing learning effectiveness

Importantly, this chapter also presented solutions and lessons learned that will benefit future developers of similar educational multiplayer games. The next chapter evaluates the completed system and presents performance metrics and assessment results.

Chapter 7

Results and Evaluation

This chapter presents the results of the HPC Sorting Serious Game development, including system features, technical performance metrics, compatibility assessment, and preliminary evaluation of educational effectiveness. The evaluation demonstrates that the project successfully met its core objectives while identifying areas for future enhancement.

7.1 Completed System Features

7.1.1 Functional Features

The implemented system includes all planned core features:

Single-Player Mode:

- Configurable number of cards (10–200)
- Random card shuffling and generation
- Drag-and-drop card manipulation
- Private buffer zones for local sorting
- Sorting validation and completion detection
- Timer and move counter
- Victory screen with performance summary

Multiplayer Mode:

- Lobby system with room creation and joining
- Support for 2–4 players
- HTTP/SSE relay-based networking

- Real-time state synchronization across clients
- Shared container visible to all players (OpenMP shared memory)
- Selective card visibility (private buffers hidden, simulating thread-private data)
- Disconnection handling and graceful degradation
- Host migration capability (partial)

User Interface:

- Main menu with mode selection
- Settings for game configuration
- In-game UI with player indicators
- Toast notifications for events
- Touch and mouse-optimized controls
- Responsive layout for different screen sizes

7.1.2 Educational Features

The game successfully implements pedagogical mappings to HPC concepts:

Table 7.1: Implemented pedagogical features

HPC Concept	Game Representation	Status
Sequential Execution	Single player mode	Implemented
Shared Memory (OpenMP)	Shared card container	Implemented
Thread-Local Storage	Private buffer zones	Implemented
Parallel Execution	Multiple players working	Implemented
Memory Access Overhead	Network latency	Implicit
Speedup	Completion time comparison	Implemented
Scalability	Variable card/player counts	Implemented
Load Balancing	Equal card distribution	Implemented
Synchronization Barriers	Turn-based phases	Partially
Race Conditions	Concurrent card access	Observable effect

7.2 Technical Performance

7.2.1 Frame Rate Performance

Frame rate was measured across representative browsers:

Table 7.2: Frame rate performance by browser

Browser	Cards	Avg FPS	Min FPS	Target
Chrome 120+ (Windows)	50	60	58	60
Chrome 120+	100	60	55	60
Chrome 120+	200	57	48	60
Firefox 120+ (Linux)	50	59	54	60
Firefox 120+	100	56	48	60
Firefox 120+	200	51	42	60
Safari 17+ (macOS)	50	60	57	60
Safari 17+	100	58	51	60
Safari 17+	200	53	44	60

Analysis:

- Target 60 FPS achieved for 50–100 cards in all browsers
- Performance degradation with 200 cards acceptable (still playable above 40 FPS)
- No significant frame drops during gameplay
- Optimization opportunities remain for very high card counts

7.2.2 Memory Usage

Memory consumption measured during typical gameplay:

Table 7.3: Memory usage by game state

Game State	Memory (MB)	Peak (MB)
Startup (Main Menu)	85	92
Single-player (50 cards)	110	125
Single-player (100 cards)	135	155
Single-player (200 cards)	175	210
Multiplayer lobby	120	140
Multiplayer game (4 players)	165	195

Analysis:

- Memory usage reasonable for modern browsers (typically several GB available)
- No memory leaks detected during extended gameplay sessions
- Object pooling successfully reduced allocation overhead

7.2.3 Web Export Size

The final web export package characteristics:

- **Uncompressed**: 28 MB
- **Gzip compressed**: 12 MB
- **Initial load time**: 3–5 seconds on typical broadband

Size Breakdown:

- Godot Engine WebAssembly runtime: 15 MB
- Game scripts and scenes: 2 MB
- Assets (textures, fonts, audio): 5 MB
- GDSync library: 1 MB

Comparison: Significantly smaller than typical Unity (50–100 MB) or Unreal Engine (100+ MB) web exports, enabling faster loading in browsers.

7.3 Platform Compatibility

7.3.1 Browser Compatibility

Testing conducted across major web browsers:

Table 7.4: Browser compatibility

Browser	Platform	Status	Notes
Chrome 120+	Windows	Excellent	Reference browser
Chrome 120+	macOS	Excellent	Full support
Firefox 120+	Windows	Excellent	Full support
Firefox 120+	Linux	Excellent	Full support
Safari 17+	macOS	Excellent	Full support
Edge 120+	Windows	Excellent	Chromium-based

Minimum Requirements:

- Modern browser with WebGL 2.0 and WebAssembly support
- 2 GB available RAM recommended
- Internet connection for multiplayer
- JavaScript enabled

7.3.2 Responsive Layout

The game adapts to various screen configurations:

- **Small windows (< 800px width)**: Compact layout with scrollable cards
- **Medium windows (800–1200px)**: Optimal experience, target size range
- **Large windows (> 1200px)**: Excellent, plenty of space for all elements

Input Support:

- **Mouse**: Full support with click-and-drag interactions
- **Touch**: Supported for touchscreen displays and tablets
- Responsive layout adjustment on window resize

7.4 Usability Evaluation

7.4.1 Informal User Testing

Informal usability testing conducted with 12 volunteer participants (students and colleagues):

Demographics:

- Age range: 20–35 years
- 8 with prior HPC knowledge, 4 without
- Mix of gamers and non-gamers
- All had web browser experience

Testing Protocol:

1. Brief introduction to project goals (no gameplay instruction)
2. Observation of first-time gameplay
3. Think-aloud protocol during play
4. Post-session questionnaire
5. Semi-structured interview

Post-Session Questionnaire: The questionnaire included the following items (5-point Likert scale unless noted):

1. **Usability**: "The game controls were intuitive and easy to learn."
2. **Visual Clarity**: "Card numbers and game elements were clearly visible."
3. **Engagement**: "I found the game engaging and would play again."
4. **Conceptual Connection**: "The game helped me understand parallel computing concepts." (HPC participants only)
5. **Multiplayer Experience**: "Coordinating with other players felt natural."
6. **Performance**: "The game ran smoothly without lag or delays."
7. **Educational Value**: "I would recommend this game to others learning about HPC."
8. **Open Response**: "What aspects of parallel computing did you recognize during gameplay?"
9. **Open Response**: "What improvements would you suggest?"

7.4.2 Key Findings

Positive Feedback:

- **Intuitive Controls**: 11/12 participants understood drag-and-drop immediately without instruction
- **Visual Clarity**: Card numbers and states were clearly visible
- **Conceptual Connection**: Participants with HPC background (8/8) recognized OpenMP shared-memory parallels
- **Engagement**: Average play session 15–20 minutes, indicating good engagement
- **Performance**: No complaints about lag or performance issues

Areas for Improvement:

- **Onboarding**: 6/12 participants wanted brief tutorial or tooltips
- **Buffer Purpose**: 5/12 weren't immediately sure why buffers were useful
- **Multiplayer Clarity**: 7/12 found it unclear what other players were doing
- **Educational Context**: Participants without HPC background (4/4) didn't make HPC connections without explanation

7.4.3 Suggested Improvements

Based on feedback, future versions should include:

1. **Optional Tutorial**: Brief interactive tutorial explaining mechanics
2. **Tooltips and Hints**: Contextual help for first-time users
3. **Player Activity Indicators**: Show what other players are doing
4. **Educational Overlay**: Optional explanations connecting gameplay to HPC concepts
5. **Strategy Hints**: Suggestions for efficient sorting approaches
6. **Replay/Analysis Mode**: Review completed games to understand performance

7.5 Educational Effectiveness

7.5.1 Preliminary Assessment

While comprehensive educational evaluation is beyond the scope of this thesis, preliminary indicators suggest educational value:

Quantitative HPC Metrics: To evaluate educational effectiveness, we introduced HPC performance concepts during gameplay:

- **Speedup (S)**: Defined as $S = T_1/T_p$ where T_1 is single-player completion time and T_p is p-player completion time. Participants observed that adding players reduced completion time, but not linearly—demonstrating Amdahl's Law in practice.

- **Efficiency (E)**: Calculated as $E = S/p = T_1/(p \cdot T_p)$. Players noted efficiency decreased with more players due to coordination overhead (communication, waiting for others).
- **Communication Overhead**: Time spent passing cards between players (analogous to message passing cost in distributed systems).
- **Load Imbalance**: Observable when one player receives more difficult cards or falls behind, reducing overall parallel efficiency.

These metrics were explained post-gameplay to connect the intuitive experience with formal HPC terminology.

Conceptual Understanding (HPC-Experienced Participants): Post-gameplay interviews revealed that participants with prior HPC knowledge:

- **8/8** recognized single-player mode as shared-memory analogy
- **7/8** identified multiplayer mode as distributed-memory simulation
- **6/8** connected buffers to thread-local storage or private memory
- **7/8** noted communication overhead when exchanging cards
- **5/8** discussed load balancing when cards were unevenly distributed
- **6/8** understood speedup concept after brief explanation
- **5/8** could relate efficiency degradation to their gameplay experience

Conceptual Introduction (Non-HPC Participants): Participants without HPC background:

- **4/4** understood basic parallelism concept (multiple players working simultaneously)
- **3/4** recognized that coordination becomes harder with more players
- **2/4** understood communication trade-offs (passing cards takes time)
- After brief explanation, **4/4** expressed interest in learning more about parallel computing

Engagement and Motivation:

- **10/12** participants expressed willingness to play again
- **9/12** would recommend to peers learning HPC
- **7/12** found it more engaging than reading textbook explanations
- **11/12** appreciated the hands-on, interactive approach

7.5.2 Pedagogical Value Proposition

The game serves as an effective **icebreaker or introductory activity**:

- **Pre-Lecture Activity**: Introduce concepts before formal instruction
- **Discussion Starter**: Generate questions and curiosity about parallel computing
- **Concept Visualization**: Provide mental model for abstract concepts
- **Reinforcement Tool**: Practice and reinforce learned concepts
- **Assessment Aid**: Reveal misconceptions for instructors to address

7.5.3 Comparison with Traditional Methods

Informal comparison with traditional HPC teaching methods (based on participant feedback):

Table 7.5: Perceived advantages over traditional methods

Criterion	**Game**	**Lecture/Textbook**
Engagement	High	Medium-Low
Immediate Feedback	Yes	No
Hands-On Experience	Yes	No
Conceptual Visualization	Strong	Weak
Detailed Explanation	Weak	Strong
Accessibility	High	Medium
Scalability (student count)	High	High
Time Required	15–30 min	60–90 min (lecture)

Conclusion: The game is not a replacement for traditional instruction but a valuable complementary tool that enhances engagement and provides intuitive understanding before formal study.

7.6 Comparison with Initial Requirements

7.6.1 Requirements Fulfillment

Assessment of how well the completed system meets initial requirements:

Table 7.6: Requirements fulfillment status

Category	Requirement	Status
Educational	Sequential baseline	✓Met
	OpenMP simulation	✓Met
	Performance feedback	✓Met
	Scalable difficulty	✓Met
	Conceptual clarity	✓Mostly
Functional	Card management	✓Met
	Single-player mode	✓Met
	Multiplayer mode	✓Met
	User interface	✓Met
	Feedback systems	✓Met
Quality	Performance (60 FPS)	✓Mostly
	Usability	✓Met
	Reliability	✓Met
	Browser Support	✓Met
	Maintainability	✓Met
	Accessibility	✓Met

Notes:

- **Conceptual Clarity**: Strong for HPC-experienced users; needs educational overlay for beginners
- **Performance**: 60 FPS achieved for typical use cases (50–100 cards); slight degradation with 200 cards
- **Browser Support**: Modern browsers fully supported across platforms

7.7 Known Limitations

7.7.1 Current Limitations

1. **Platform Support**: Currently web-only; native mobile apps not implemented
2. **Educational Scaffolding**: Limited in-game explanations; requires instructor context
3. **Sorting Algorithms**: Only supports general sorting; doesn't demonstrate specific parallel algorithms (merge sort, sample sort)
4. **Performance Metrics**: Basic timing and moves; lacks detailed profiling (speedup curves, scalability graphs)

5. **Advanced HPC Concepts**: Doesn't cover race conditions, deadlocks, synchronization primitives explicitly
6. **Automated Assessment**: No built-in assessment or progress tracking for educational use
7. **Formal Evaluation**: Lacks comprehensive educational efficacy study

7.7.2 Scope Limitations

Certain features were intentionally deferred to maintain project scope:

- User accounts and progress persistence
- Leaderboards and social features
- Advanced sorting algorithm demonstrations
- GPU parallelism (CUDA/OpenCL) simulations
- Instructor dashboard for classroom management
- Detailed analytics and learning analytics integration

7.8 Summary

This chapter presented comprehensive evaluation results:

- **Completed Features**: All core functional and educational features successfully implemented
- **Technical Performance**: Meets or exceeds performance targets on representative devices
- **Platform Compatibility**: Excellent browser compatibility across platforms
- **Usability**: Generally positive usability feedback with actionable improvement suggestions
- **Educational Effectiveness**: Preliminary evidence suggests value as introductory/supplementary tool
- **Requirements Fulfillment**: Nearly all initial requirements met or exceeded
- **Limitations**: Known limitations documented for transparency and future work planning

The results demonstrate that the project successfully achieved its core objective: creating a functional, engaging serious game for teaching HPC concepts on the web platform. The next chapter concludes the thesis and discusses future directions for research and development.

Chapter 8

Conclusion and Future Work

This final chapter summarizes the contributions of this thesis, reflects on how the research questions were answered, discusses implications for HPC education and serious game development, and outlines comprehensive directions for future work.

8.1 Summary of Contributions

This thesis presented the design, implementation, and evaluation of the HPC Sorting Serious Game, a web-first educational tool for teaching parallel computing concepts through interactive card-sorting gameplay.

8.1.1 Primary Contributions

The main contributions of this work include:

1. Novel Pedagogical Approach

- Digital transformation of Professor D'Agostino's successful physical classroom experiments into a scalable, accessible web-based application
- Systematic mapping of HPC concepts (specifically OpenMP shared-memory parallelism and sequential vs. parallel execution) to concrete, manipulable game mechanics
- Demonstration that serious games can serve as effective icebreakers for teaching abstract parallel computing concepts

2. Technical Implementation

- Functional serious game prototype demonstrating real-time multiplayer synchronization on mobile platforms
- Solutions to multiplayer state management challenges, particularly selective visibility and host-authoritative architecture

- Integration of GDSync framework with WebRTC for peer-to-peer educational gaming
- Mobile UI/UX design patterns for displaying and manipulating numerous interactive elements on small screens

3. Documentation and Knowledge Transfer

- Comprehensive documentation of technical challenges (especially GDSync framework issues) and solutions
- Identification of design patterns and best practices for educational multiplayer game development
- Open-source codebase (MIT license) enabling reuse, extension, and further research
- Contributions to GDSync project documentation and issue resolution

4. Preliminary Validation

- Demonstration of technical feasibility through performance metrics and compatibility testing
- Positive preliminary usability feedback from informal user testing
- Evidence suggesting educational value as supplementary/introductory tool for HPC education

8.2 Research Questions Revisited

Returning to the research problem stated in Chapter 1:

> *How can we design and implement an effective serious game for web platforms that teaches fundamental High-Performance Computing concepts (specifically OpenMP shared-memory parallelism and sequential vs. parallel execution) through interactive card-sorting gameplay while overcoming the technical challenges of multiplayer state synchronization and responsive UI/UX constraints?*

8.2.1 Addressing the Educational Problem

Q: How can abstract parallel computing concepts be mapped to concrete, understandable game mechanics?

A: The thesis demonstrated successful mapping through:

- Card-sorting as metaphor for data manipulation
- Shared container representing shared memory (OpenMP)
- Private buffers representing thread-local storage (OpenMP)
- Card movement between shared container and private buffers representing memory access patterns
- Network latency representing synchronization and communication overhead
- Timer and move counter representing performance metrics

This mapping was validated through recognition by HPC-experienced participants and engagement by HPC-novice participants.

8.2.2 Addressing the Technical Problem

Q: How can we implement real-time multiplayer gameplay on mobile devices with acceptable performance?

A: The thesis demonstrated that:

- WebRTC peer-to-peer networking eliminates server costs while providing low latency
- GDSync framework (despite challenges) simplifies state synchronization
- Host-authoritative architecture balances consistency and responsiveness
- Godot Engine provides adequate performance for 2D card-based gameplay
- Target 60 FPS achieved for typical scenarios on mid-range devices

8.2.3 Addressing the Design Problem

Q: How can we balance educational effectiveness with engagement and playability on mobile platforms?

A: The thesis showed that:

- Touch-based drag-and-drop provides intuitive, engaging interaction
- Visual clarity and immediate feedback support learning
- Scalable difficulty accommodates learners at different levels
- Game mechanics naturally lead to questions about parallel computing

- Informal testing suggests good engagement without sacrificing educational goals

However, the design can be improved with better onboarding and educational scaffolding for learners without prior HPC exposure.

8.3 Implications

8.3.1 For HPC Education

This work has several implications for teaching parallel computing:

Complementary Tool, Not Replacement: Serious games are most valuable as **complementary tools** alongside traditional instruction:

- Use as pre-lecture icebreaker to introduce concepts
- Employ as reinforcement after formal instruction
- Leverage to uncover student misconceptions
- Deploy as discussion starter for deeper exploration

Accessibility Matters: Web-first design significantly enhances accessibility:

- No specialized HPC hardware required
- No app installation needed—runs directly in browsers
- Students can access from any device with a web browser
- Cross-platform compatibility (Windows, macOS, Linux, mobile)
- Reduces barriers to entry for HPC education

Visualization Is Powerful: Making abstract concepts visible and manipulable:

- Helps students build intuitive mental models
- Facilitates understanding before mathematical formalization
- Engages different learning styles
- Makes HPC concepts less intimidating

8.3.2 For Serious Game Development

This work offers insights for developers of educational multiplayer games:

Framework Selection is Critical:

- Thoroughly evaluate frameworks before committing
- Test advanced use cases early in development
- Build abstraction layers to facilitate future framework changes
- Engage with framework maintainers proactively

Multiplayer Adds Complexity:

- Start with single-player; add multiplayer incrementally
- Host-authoritative model simplifies consistency management for educational games
- Debugging multiplayer requires specialized tools and strategies
- Network variability must be designed for, not retrofitted

Mobile Constraints Are Real:

- Design for small screens and touch input from the start
- Performance optimization is essential, not optional
- Battery life and data usage are important considerations
- Test on diverse devices, not just flagship models

8.4 Future Work

Numerous opportunities exist for extending and improving this work.

8.4.1 Short-Term Enhancements

Improvements that could be implemented in 1–3 months:

Onboarding and Tutorial:

- Interactive tutorial explaining game mechanics
- Tooltips and contextual hints for first-time users
- Guided first playthrough highlighting key concepts

Educational Overlay:

- Optional explanations connecting gameplay to HPC concepts
- Pop-up definitions of terms (thread, process, synchronization)
- Post-game summary explaining what was learned
- Links to additional resources for deeper learning

Enhanced Feedback:

- Player activity indicators showing what others are doing
- Visual representation of communication patterns
- Performance analysis comparing strategies
- Suggestions for improvement

iOS Support:

- Export to iOS using Godot's iOS templates
- Address Apple App Store requirements
- Test on iPhone and iPad devices

8.4.2 Medium-Term Extensions

Enhancements requiring 3–6 months of development:

Additional Game Modes:

1. **Algorithm-Specific Modes**: Demonstrate specific parallel sorting algorithms
   - Parallel merge sort
   - Sample sort (distributed-memory style, future MPI extension)
   - Bitonic sort
2. **Advanced Concepts**: Introduce more HPC topics
   - Synchronization barriers and critical sections
   - Race conditions and deadlock scenarios
   - Load balancing challenges
   - Communication patterns (broadcast, scatter/gather, reduce)
3. **Competitive Modes**: Enhance engagement
   - Time trials with leaderboards
   - Team vs. team competitions
   - Challenge modes with specific constraints

Enhanced Analytics:

- Detailed performance metrics (speedup, efficiency, scalability)
- Strategy analysis and optimization suggestions
- Comparison with theoretical optimal performance
- Progress tracking across multiple sessions

Web Export:

- HTML5 export for browser-based play
- Integration with learning management systems (Moodle, Canvas)
- Embeddable widget for course websites
- Cross-platform multiplayer (mobile + web)

Instructor Dashboard:

- Classroom management interface
- Real-time monitoring of student gameplay
- Assessment and analytics integration
- Customizable game parameters and scenarios

8.4.3 Long-Term Research Directions

Research directions requiring 6+ months and potential collaborations:

Formal Educational Evaluation: Conduct rigorous educational research to assess learning outcomes:

- **Study Design**: Pre/post-test controlled study with treatment and control groups
- **Participants**: Undergraduate students in HPC/parallel computing courses
- **Measures**: Knowledge assessments, concept inventories, attitude surveys
- **Analysis**: Statistical comparison of learning gains between groups
- **Publication**: Results submitted to CS education conferences (SIGCSE, ICER)

Learning Analytics Integration:

- Instrument game to collect detailed interaction data
- Analyze gameplay patterns to identify learning difficulties
- Develop predictive models for student understanding
- Personalize game difficulty based on demonstrated mastery
- Research publication in learning analytics venues (LAK, EDM)

GPU Parallelism Extension: Extend game to cover GPU computing concepts (CUDA, OpenCL):

- Massive parallelism simulation (thousands of "mini-workers")
- Warp/wavefront concept representation
- Memory hierarchy visualization (global, shared, local)
- Divergence and bank conflict demonstrations

Distributed Memory Extension (MPI): Future extension to simulate MPI distributed-memory programming:

- Separate player "nodes" with isolated memory spaces
- Explicit message passing between nodes (simulating MPI_Send/MPI_Recv)
- Support for hybrid MPI+OpenMP programming models
- Hierarchical parallelism with shared memory within nodes
- Load balancing across heterogeneous resources

AI-Assisted Learning: Integrate AI to enhance educational effectiveness:

- AI opponent for single-player mode
- Intelligent tutoring system providing hints
- Automatic difficulty adjustment
- Natural language explanations of concepts

8.4.4 Community and Ecosystem Development

Open Source Community Building:

- Publish to GitHub with comprehensive documentation
- Create contributor guidelines and issue templates
- Engage educational computing communities
- Accept contributions from students and educators
- Maintain active development and support

Educational Resource Development:

- Instructor guides with lesson plans
- Student worksheets and reflection prompts
- Video tutorials and gameplay examples
- Integration guides for popular LMS platforms
- Curriculum modules incorporating the game

Conference and Journal Publications: Disseminate findings to relevant research communities:

- **CS Education**: SIGCSE, ICER, ITiCSE
- **HPC Education**: EduHPC, XSEDE Education Track
- **Serious Games**: Games+Learning+Society, FDG
- **Educational Technology**: Learning @ Scale, EC-TEL
- **HPC Venues**: SC Education Program, ISC tutorials

Potential paper topics:

1. Design and implementation of mobile serious game for HPC education
2. Evaluation of learning outcomes with serious game intervention
3. Technical challenges and solutions in educational multiplayer games
4. Pedagogical mapping from physical to digital activities

8.5 Limitations and Reflections

8.5.1 Methodological Limitations

8.5.1.1 Evaluation Scope:

- Informal usability testing with small sample (n=12)
- No formal educational efficacy study
- Limited diversity in participant demographics
- Short-term evaluation only (no long-term retention studies)

8.5.1.2 Technical Scope:

- Single platform implementation (Android only)
- Focus on basic parallel concepts (no advanced topics)
- Simplified metaphors (not high-fidelity simulations)
- Limited integration with formal educational contexts

8.5.2 Reflections on the Development Process

What Worked Well:

- Iterative development allowed course correction
- Open-source toolchain reduced costs and increased flexibility
- Community engagement provided valuable support
- Documentation-first approach facilitated thesis writing
- Informal testing surfaced usability issues early

What Could Be Improved:

- Earlier, more frequent testing with target users
- More thorough framework evaluation before commitment
- Formal project management and milestone tracking
- Automated testing infrastructure from the start
- More structured approach to educational validation

8.6 Concluding Remarks

This thesis presented the design, implementation, and evaluation of the HPC Sorting Serious Game, demonstrating that:

1. **Serious games can effectively teach HPC concepts** through intuitive, engaging mechanics that transform abstract ideas into concrete, manipulable interactions.
2. **Mobile platforms are viable for educational multiplayer games**, despite technical challenges in state synchronization, UI/UX design, and performance optimization.
3. **Open-source technologies enable accessible HPC education** by eliminating cost barriers and empowering educators to customize and extend educational tools.
4. **Interdisciplinary challenges inspire innovative solutions** at the intersection of computer science education, game design, parallel computing, and mobile development.

The HPC Sorting Serious Game represents a step toward making High-Performance Computing education more accessible, engaging, and effective. By leveraging the ubiquity of smartphones and the motivational power of games, this work contributes to democratizing access to advanced computing concepts.

As parallel computing becomes increasingly fundamental to modern software engineering—from multicore CPUs to cloud computing to AI/ML workloads—innovative educational approaches are essential to prepare the next generation of computing professionals. Serious games, when thoughtfully designed and rigorously evaluated, can play a valuable role in this educational mission.

The author hopes this work inspires further research and development in educational technologies for HPC and serves as a useful resource for educators, students, and game developers interested in the serious games domain.

8.7 Final Thoughts

$$Tobewrittenlast, afterallrevisionsarecomplete.$$

Teaching parallel computing remains challenging, but it need not be boring or inaccessible. By combining pedagogical insight, technical innovation, and game design principles, we can create learning experiences that are both effective and enjoyable.

The journey from physical card-sorting experiments in a classroom to a digital multiplayer mobile game demonstrates the potential for technology to scale and extend proven teaching methods. While the game developed in this thesis is far from perfect, it represents a meaningful contribution to the ongoing effort to make HPC education more engaging and accessible.

As we look to the future, the convergence of mobile computing, cloud infrastructure, and educational research offers exciting possibilities for serious games in computing education. The author looks forward to seeing how this work evolves, inspires further innovations, and ultimately contributes to better outcomes for students learning parallel computing.

"The best way to learn is by doing."
— Ancient Proverb

Bibliography

[1] M. Zyda, "From visual simulation to virtual reality to games," *Computer*, vol. 38, no. 9, pp. 25–32, 2005.

[2] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. New York: Harper & Row, 1990.

[3] T. M. Connolly, E. A. Boyle, E. MacArthur, T. Hainey, and J. M. Boyle, "A systematic literature review of empirical evidence on computer games and serious games," *Computers & Education*, vol. 59, no. 2, pp. 661–686, 2012.

[4] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[5] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

Appendix A

User Manual

A.1 Introduction

This user manual provides comprehensive instructions for installing, configuring, and playing the HPC Sorting Serious Game. It is intended for students, educators, and anyone interested in learning about parallel computing through interactive gameplay.

A.2 System Requirements

A.2.1 Minimum Requirements

- **Operating System**: Android 7.0 (API level 24) or higher
- **RAM**: 2 GB minimum
- **Storage**: 50 MB free space
- **Screen**: 5-inch display or larger
- **Graphics**: OpenGL ES 3.0 support
- **Network**: WiFi or mobile data (for multiplayer mode only)

A.2.2 Recommended Requirements

- Android 9.0 or higher
- 4 GB RAM
- 5.5–6.5 inch display
- Stable WiFi connection for multiplayer

A.3 Installation

A.3.1 Installing from APK

1. Download the APK file from the official repository or distribution site
2. Navigate to the APK file using your device's file manager
3. Tap the APK file to begin installation
4. If prompted, allow installation from unknown sources in Settings
5. Follow the on-screen installation prompts
6. Launch the app from your app drawer

A.3.2 Installing from Google Play (Future)

Once published to Google Play Store:

1. Open Google Play Store app
2. Search for "HPC Sorting Serious Game"
3. Tap "Install"
4. Wait for download and installation to complete
5. Tap "Open" to launch

A.4 Getting Started

A.4.1 First Launch

On first launch, you will see the main menu with options:

- **Single Player**: Practice sorting alone (sequential execution)
- **Multiplayer**: Join or host a game with others (OpenMP shared-memory simulation)
- **Settings**: Configure game options
- **About**: Information about the game and credits

A.5 Single-Player Mode

A.5.1 Starting a Single-Player Game

1. Tap "Single Player" from the main menu
2. Configure game settings:
   - Number of cards: 10–200 (recommended: 50 for beginners)
   - Difficulty: Easy, Medium, Hard
3. Tap "Start Game"

A.5.2 Gameplay

A.5.2.0.1 Objective: Sort all cards in ascending order in the main container as quickly as possible.

A.5.2.0.2 Controls:

- **Drag**: Long-press on a card and drag to move it
- **Drop**: Release finger to drop card in desired location
- **Scroll**: Swipe vertically to scroll through cards
- **Zoom**: Pinch to zoom in/out (if enabled)

A.5.2.0.3 Using Buffers:

- Each player has 2–3 private buffer zones
- Drag cards into buffers for temporary storage
- Sort cards within buffers before merging back
- Buffers simulate thread-local storage in parallel computing

A.5.2.0.4 Winning:

- Game completes when all cards are in correct ascending order
- Victory screen shows completion time and number of moves
- Try to minimize both time and moves for better performance

A.6 Multiplayer Mode

A.6.1 Creating a Game

1. Tap "Multiplayer" from the main menu
2. Tap "Create Game"
3. Configure settings:
   - Number of players: 2–4
   - Number of cards: 10–200
   - Game mode: Cooperative or Competitive
4. A unique room code will be generated
5. Share this code with other players
6. Wait for players to join
7. Once all players are ready, tap "Start Game"

A.6.2 Joining a Game

1. Tap "Multiplayer" from the main menu
2. Tap "Join Game"
3. Enter the room code provided by the host
4. Tap "Join"
5. Wait in the lobby for the host to start the game

A.6.3 Multiplayer Gameplay

A.6.3.0.1 Shared Memory Access:

- All cards are visible in the shared container (simulating shared memory)
- Each player has private buffers for local work (thread-local storage)
- Players can access the shared container simultaneously
- Other players' private buffers are not directly visible (simulating thread-private data)

A.6.3.0.2 Collaboration:

- Players must coordinate to sort all cards
- Cards can be passed between players (message passing)
- Communication and strategy are key to success
- Network latency simulates communication overhead in HPC

A.6.3.0.3 Winning Together:

- Team wins when all cards across all players are sorted
- Final merge phase combines sorted sequences
- Victory screen shows team performance metrics

A.7 Settings

A.7.1 Game Settings

- **Default Card Count**: Set preferred number of cards for quick start
- **Difficulty**: Choose default difficulty level
- **Theme**: Light or dark mode

A.7.2 Audio Settings

- **Sound Effects**: Enable/disable gameplay sounds
- **Volume**: Adjust volume level

A.7.3 Display Settings

- **Card Size**: Adjust card display size
- **Orientation**: Portrait, landscape, or auto
- **Visual Effects**: Enable/disable animations

A.7.4 Network Settings

- **Connection Type**: WiFi preferred, mobile data, or either
- **Signaling Server**: Configure custom server (advanced)

A.8 Tips and Strategies

A.8.1 Efficient Sorting Strategies

1. **Divide and Conquer**: Use buffers to sort smaller subsets of cards
2. **Merge Sorted Sequences**: Maintain sorted order within buffers, then merge
3. **Minimize Moves**: Think before moving; avoid unnecessary rearrangements
4. **Identify Patterns**: Look for already-sorted subsequences

A.8.2 Multiplayer Coordination

1. **Communicate**: Use external communication (voice, text) to coordinate
2. **Divide Responsibility**: Assign value ranges to each player
3. **Minimize Passing**: Reduce card exchanges to reduce overhead
4. **Plan Ahead**: Discuss strategy before starting

A.9 Troubleshooting

A.9.1 Common Issues and Solutions

A.9.1.0.1 Game Won't Launch:

- Verify Android version meets minimum requirements
- Ensure sufficient free storage space
- Try restarting device
- Reinstall the app if necessary

A.9.1.0.2 Performance Issues (Lag):

- Reduce number of cards
- Disable visual effects in settings
- Close background apps
- Ensure device is not overheating

A.9.1.0.3 Cannot Connect to Multiplayer:

- Verify internet connection is active
- Check that both players are using the same game version
- Try switching between WiFi and mobile data
- Verify firewall/network settings allow connections
- Re-enter room code carefully

A.9.1.0.4 Multiplayer Lag or Disconnections:

- Switch to more stable network connection
- Move closer to WiFi router
- Reduce number of players
- Check that signaling server is reachable

A.10 Educational Use

A.10.1 For Students

- Play single-player mode first to understand mechanics
- Observe how buffers help organize your sorting work
- Notice how coordination becomes harder with more players
- Reflect on how game concepts relate to parallel programming
- Experiment with different strategies and compare performance

A.10.2 For Instructors

- Use as pre-lecture icebreaker to introduce HPC concepts
- Facilitate discussion after gameplay about parallel computing
- Ask students to identify real-world parallels (threads, shared memory)
- Connect game mechanics to OpenMP programming model and sequential execution
- Use performance metrics to discuss speedup and scalability

A.11 Support and Resources

A.11.1 Getting Help

- **GitHub Repository**: `https://github.com/[username]/hpc-sorting-serious-game`
- **Issue Tracker**: Report bugs and request features on GitHub Issues
- **Documentation**: Additional documentation in the repository wiki
- **Contact**: Szymon Zinkowicz - `[email]`

A.11.2 Contributing

The project is open-source (MIT license). Contributions are welcome:

- Report bugs and suggest features via GitHub Issues
- Submit pull requests with improvements
- Contribute documentation and educational resources
- Share feedback from classroom use

Appendix B

Code Listings

B.1 Source Code Repository

The complete source code for the HPC Sorting Serious Game is available as an open-source project:

- **Repository**: https://github.com/szymonzinkowicz/hpc-sorting-serious-game
- **License**: MIT License
- **Language**: GDScript (Godot 4.x)
- **Documentation**: README and inline comments

B.2 Project Structure

The project follows Godot Engine conventions:

```
hpc-sorting-serious-game/
+-- project.godot         # Project configuration
+-- scenes/               # Scene files (.tscn)
|   +-- main_menu.tscn
|   +-- singleplayer_game.tscn
|   +-- multiplayer_game.tscn
|   +-- components/       # Reusable UI components
+-- scripts/              # GDScript source files
|   +-- card.gd           # Card logic
|   +-- card_manager.gd   # Card generation/shuffling
|   +-- game_manager.gd   # Game state management
|   +-- networking/       # GDSync integration
+-- assets/               # Textures, fonts, audio
+-- addons/               # GDSync plugin
+-- export_presets.cfg    # Web export configuration
```

B.3 Key Components

B.3.1 Card System

The card system handles drag-and-drop interaction and visual state:

- `card.gd`: Individual card with value, drag handling, visual feedback
- `card_manager.gd`: Generation, shuffling, sorting validation
- `card_container.gd`: Drop zones (shared container, private buffers)

B.3.2 Networking

Multiplayer functionality uses GDSync with HTTP/SSE transport:

- `lobby_manager.gd`: Room creation, joining, player management
- `state_sync.gd`: Card movement synchronization across clients
- `http_transport.gd`: Custom transport layer for web compatibility

B.3.3 Game Logic

Core game mechanics:

- `game_manager.gd`: Mode selection, timer, win condition checking
- `sorting_validator.gd`: Checks if cards are in correct order
- `performance_tracker.gd`: Measures time, moves for metrics

B.4 Configuration

B.4.1 Web Export Settings

Key settings in `export_presets.cfg` for web deployment:

- **Export Mode**: WebAssembly (WASM)
- **Threads**: Single-threaded (for browser compatibility)
- **VRAM Compression**: Disabled (better compatibility)
- **Canvas Resize Policy**: Responsive

B.4.2 GDSync Configuration

The HTTP/SSE relay server URL and settings are configured in the project autoload.

B.5 Building and Running

B.5.1 Development

1. Install Godot Engine 4.x
2. Clone the repository
3. Open `project.godot` in Godot Editor
4. Press F5 to run locally

B.5.2 Web Export

1. In Godot: Project → Export
2. Select "Web" preset
3. Click "Export Project"
4. Deploy `index.html` and assets to web server

B.5.3 Relay Server

The HTTP/SSE relay server (Python/aiohttp) is in the `signaling-server/` directory. See Appendix **??** for API documentation.

Appendix C

API Documentation

C.1 Introduction

This appendix documents the key APIs and interfaces used in the HPC Sorting Serious Game, including custom classes, GDSync integration, and plugin APIs. This documentation is intended for developers who wish to extend or modify the game.

C.2 Core Game Classes

C.2.1 Card Class

The Card class represents an individual playing card.

C.2.1.0.1 Properties:

- `card_value: int` - The numeric value displayed on the card
- `card_id: int` - Unique identifier for networking synchronization
- `is_dragging: bool` - Whether the card is currently being dragged
- `is_selected: bool` - Whether the card is selected
- `original_position: Vector2` - Position before drag operation

C.2.1.0.2 Methods:

- `set_card_value(value: int)` - Sets the card's displayed value
- `get_card_value() -> int` - Returns the card's value
- `start_drag()` - Initiates drag operation
- `end_drag()` - Completes drag operation
- `update_display()` - Refreshes visual representation

C.2.1.0.3 Signals:

- `card_selected(card: Card)` - Emitted when card is selected
- `card_dragged(card: Card, position: Vector2)` - Emitted during drag
- `card_dropped(card: Card, target: Node)` - Emitted on drop

C.2.2 CardManager Class

Manages the collection of cards and game logic.

C.2.2.0.1 Properties:

- `cards: Array[Card]` - Array of all cards in the game
- `card_id_counter: int` - Counter for unique card IDs

C.2.2.0.2 Methods:

- `generate_cards(count: int) -> Array[Card]` - Creates specified number of cards
- `shuffle_cards()` - Randomly shuffles the card array
- `is_sorted(card_array: Array) -> bool` - Checks if array is sorted
- `check_global_sorted() -> bool` - Checks if all cards are sorted
- `get_card_by_id(id: int) -> Card` - Retrieves card by ID

C.2.2.0.3 Signals:

- `cards_generated(count: int)` - Emitted after card generation
- `cards_shuffled()` - Emitted after shuffling
- `sorting_complete()` - Emitted when sorting is finished

C.2.3 Buffer Class

Represents a temporary storage area for cards (thread-local storage simulation).

C.2.3.0.1 Properties:

- `buffer_id: int` - Unique identifier
- `owner_id: int` - ID of owning player (-1 for shared)
- `contained_cards: Array[Card]` - Cards currently in buffer
- `max_capacity: int` - Maximum number of cards (optional)

C.2.3.0.2 Methods:

- `accept_card(card: Card) -> bool` - Attempts to add card
- `remove_card(card: Card)` - Removes card from buffer
- `get_cards() -> Array[Card]` - Returns all contained cards
- `is_full() -> bool` - Checks if buffer is at capacity
- `sort_contents()` - Sorts cards within buffer

C.2.3.0.3 Signals:

- `card_added(card: Card)` - Emitted when card added
- `card_removed(card: Card)` - Emitted when card removed
- `buffer_sorted()` - Emitted after sorting

C.3 Multiplayer Classes

C.3.1 LobbyManager Class

Manages multiplayer lobby functionality.

C.3.1.0.1 Properties:

- `players: Dictionary` - Map of player_id to player_info
- `is_host: bool` - Whether local player is host
- `room_code: String` - Unique room identifier

C.3.1.0.2 Methods:

- `create_lobby() -> String` - Creates lobby, returns room code
- `join_lobby(code: String) -> bool` - Joins existing lobby
- `add_player(player_id: int, player_name: String)` - Adds player to lobby
- `remove_player(player_id: int)` - Removes player from lobby
- `start_game()` - Initiates game start (host only)

C.3.1.0.3 RPCs:

- `@rpc rpc_player_joined(player_id: int, player_name: String)` - Broadcasts player join
- `@rpc rpc_player_left(player_id: int)` - Broadcasts player leave
- `@rpc rpc_start_game()` - Signals game start to all clients

C.3.1.0.4 Signals:

- `player_joined(player_id: int, player_name: String)` - Player joined
- `player_left(player_id: int)` - Player left
- `game_started()` - Game has started

C.3.2 NetworkSync Class

Handles state synchronization across clients.

C.3.2.0.1 Properties:

- `card_ownership: Dictionary` - Maps card_id to owner_player_id
- `game_state: Dictionary` - Global game state data

C.3.2.0.2 Methods:

- `@GDSync.rpc sync_move_card(card_id, target_id, position)` - Syncs card movement
- `@GDSync.rpc request_move_card(card_id, target_id)` - Requests validated move
- `update_card_visibility()` - Updates visibility based on ownership
- `broadcast_game_state()` - Sends full state to new clients

C.4 GDSync Framework API

C.4.1 Sync Annotations

GDSync provides annotations for automatic synchronization:

C.4.1.0.1 @GDSync.sync

Marks a variable or node for automatic synchronization.

```
# Synchronize a variable
@GDSync.sync(mode=GDSync.SYNC_MODE.AUTHORITY)
var player_position: Vector2

# Synchronize a node
@GDSync.sync(mode=GDSync.SYNC_MODE.REPLICATED)
var replicated_node: Node
```

C.4.1.0.2 Sync Modes:

- `AUTHORITY` - Only authority (usually host) can modify
- `REPLICATED` - Replicated to all peers
- `OWNER` - Only owner can modify

C.4.2 RPC Annotations

C.4.2.0.1 @GDSync.rpc

Marks a function for remote procedure call.

```
# RPC callable by any peer
@GDSync.rpc(call_local=true)
func my_networked_function(arg1: int, arg2: String):
    # Function body
    pass
```

```

# Authority-only RPC
@GDSync.rpc(call_local=false, authority_only=true)
func host_only_function():
    # Only host can call this
    pass
```

C.4.2.0.2 Parameters:

- `call_local: bool` - Whether to also execute locally
- `authority_only: bool` - Restrict to authority
- `reliable: bool` - Use reliable transmission

C.4.3 Connection Management

C.4.3.0.1 Methods:

- `GDSync.create_server(port: int)` - Creates server
- `GDSync.connect_to_server(address: String, port: int)` - Connects to server
- `GDSync.disconnect()` - Disconnects from network
- `GDSync.get_peers() -> Array` - Returns list of connected peers

C.4.3.0.2 Signals:

- `peer_connected(peer_id: int)` - Peer connected
- `peer_disconnected(peer_id: int)` - Peer disconnected
- `connection_failed()` - Connection attempt failed

C.5 Plugin APIs

C.5.1 ToastParty API

C.5.1.0.1 Methods:

- `ToastParty.show_toast(message: String, duration: int)`
- `ToastParty.show_toast_at_position(message: String, position: Vector2, duration: int)`

C.5.1.0.2 Constants:

- `LENGTH_SHORT` - 2 seconds
- `LENGTH_LONG` - 5 seconds
- `TYPE_INFO` - Informational toast
- `TYPE_SUCCESS` - Success toast (green)
- `TYPE_ERROR` - Error toast (red)
- `TYPE_WARNING` - Warning toast (yellow)

C.5.2 Logger API

C.5.2.0.1 Methods:

- `Logger.debug(message: String)` - Debug-level logging
- `Logger.info(message: String)` - Info-level logging
- `Logger.warn(message: String)` - Warning-level logging
- `Logger.error(message: String)` - Error-level logging

C.5.2.0.2 Configuration:

- `Logger.set_log_level(level: int)` - Set minimum log level
- `Logger.enable_file_logging(path: String)` - Log to file
- `Logger.add_filter(tag: String)` - Filter log messages

C.6 Utility APIs

C.6.1 Profiler API

C.6.1.0.1 Methods:

- `Profiler.start_timer(label: String)` - Start timing
- `Profiler.end_timer(label: String) -> float` - End timing, return elapsed ms
- `Profiler.measure_fps() -> float` - Get current FPS
- `Profiler.get_memory_usage() -> int` - Get memory usage in bytes

C.6.2 Config Manager API

C.6.2.0.1 Methods:

- `Config.get_value(section: String, key: String, default: Variant) -> Variant`
- `Config.set_value(section: String, key: String, value: Variant)`
- `Config.save_config()` - Save configuration to disk
- `Config.load_config()` - Load configuration from disk

C.7 Event System

C.7.1 Global Events

The game uses an event bus for decoupled communication.

```
# Emit global event
EventBus.emit("game_started", {"players": 4})

# Listen for event
EventBus.connect("game_started", _on_game_started)

func _on_game_started(data: Dictionary):
    print("Game started with ", data["players"], " players")
```

C.7.1.0.2 Common Events:

- `game_started` - Game has begun
- `game_ended` - Game has concluded
- `player_joined` - Player joined lobby
- `player_left` - Player left game
- `card_moved` - Card was moved
- `sorting_complete` - Sorting finished

C.8 Extension Points

C.8.1 Custom Game Modes

To create custom game modes, extend the `GameMode` base class:

```
extends GameMode
class_name MyCustomMode

func _init():
    mode_name = "My Custom Mode"
    mode_description = "A custom game variant"

func setup_game():
    # Custom initialization logic
    pass

func validate_move(card: Card, target: Node) -> bool:
    # Custom move validation
    return true

func check_win_condition() -> bool:
    # Custom win condition
    return false
```

C.8.2 Custom Card Types

Extend the Card class for custom card behavior:

```
extends Card
class_name SpecialCard

@export var special_ability: String = ""

func activate_ability():
    match special_ability:
        "double_value":
            card_value *= 2
        "wildcard":
            # Can match any value
            pass
    update_display()
```

C.9 Development Tools

C.9.1 Debug Commands

Console commands for development:

- `/spawn_cards <count>` - Generate cards
- `/sort_all` - Automatically sort all cards
- `/reset_game` - Reset game state
- `/show_network_stats` - Display network statistics
- `/toggle_debug_ui` - Show/hide debug information

Appendix D

Study Materials

D.1 Introduction

This appendix provides educational materials to help students and instructors connect the HPC Sorting Serious Game to formal parallel computing concepts. It includes learning objectives, discussion questions, exercises, and connections to OpenMP programming and sequential vs. parallel execution.

D.2 Learning Objectives

D.2.1 Foundational Concepts

After playing the game, students should be able to:

1. **Define Parallelism**: Explain what it means to perform multiple tasks at the same time
2. **Understand Teamwork Benefits**: Describe how working together can finish tasks faster
3. **Recognize Coordination Challenges**: Explain that working together requires communication and planning
4. **Appreciate Fair Work Distribution**: Understand why it's important that everyone has about the same amount of work
5. **Identify When Teamwork Helps**: Recognize that some tasks benefit from multiple workers more than others

D.2.2 Sequential Execution Concepts (Single-Player Mode)

Students should understand:

1. A single process executes all operations sequentially

2. No parallelism or concurrency
3. Simple but potentially slower for large datasets
4. Baseline for comparison with parallel execution
5. All data is accessible directly without coordination

D.2.3 OpenMP Concepts (Multiplayer Mode)

Students should understand:

1. Multiple threads share a common memory space
2. Each thread can have private (local) data in buffers
3. Threads work collaboratively on shared data
4. Synchronization occurs through shared container access
5. Work is divided among threads for parallel execution
6. Final merge combines results from parallel execution

D.3 Discussion Questions

D.3.1 Pre-Game Questions

Use these questions before gameplay to activate prior knowledge:

1. Have you ever worked on a group project? How did you divide the work?
2. What challenges arise when multiple people work on the same task simultaneously?
3. How is sorting a deck of cards different when one person does it versus multiple people?
4. Can you think of tasks that benefit from having multiple workers? Which tasks don't?

D.3.2 Post-Game Discussion: Single-Player Mode

After single-player gameplay:

1. How did you use the buffer zones? Why were they helpful?
2. What strategy did you use to sort the cards efficiently?
3. How would your strategy change with 10 cards versus 200 cards?
4. If you could have multiple versions of yourself working simultaneously, how would you coordinate?
5. What parallels can you draw between the buffers and computer memory?

D.3.3 Post-Game Discussion: Multiplayer Mode

After multiplayer gameplay:

1. How did you communicate and coordinate with other players?
2. What happened when two players tried to work on the same cards?
3. Did you notice delays when passing cards between players? Why might that occur?
4. How did you divide the work? Was it balanced?
5. What would happen if one player had many more cards than others?
6. How is this different from single-player mode?

D.3.4 Connecting to HPC Concepts

Bridge from game to formal concepts:

1. How does single-player mode relate to shared-memory parallel programming?
2. How does multiplayer mode relate to distributed-memory parallel programming?
3. What real-world computing problems benefit from parallelism?
4. What are the trade-offs between shared-memory and distributed-memory approaches?
5. When might communication costs outweigh parallelism benefits?

D.4 Classroom Activities

D.4.1 Activity 1: Strategy Competition

D.4.1.0.1 Objective: Compare different ways to sort cards and see which works best.

D.4.1.0.2 Instructions:

1. Divide class into teams of 2–3 students
2. Each team plays the game with the same settings (e.g., 50 cards, 3 players)
3. Teams write down their strategy: How will you divide the work? Will you help each other?
4. Compare completion times between teams
5. Discuss which strategies worked well and why

D.4.1.0.3 Discussion:

- Which strategy was fastest? Why do you think it worked better?
- Did teams that divided work evenly finish faster?
- What problems did your team face during the game?
- If you played again, what would you do differently?

D.4.2 Activity 2: Does More Players Mean Faster?

D.4.2.0.1 Objective: Investigate how adding more players affects completion time.

D.4.2.0.2 Instructions:

1. Students play multiplayer mode with different numbers of players:
   - 2 players, 50 cards
   - 3 players, 50 cards
   - 4 players, 50 cards
2. Record completion times for each configuration
3. Create a simple chart showing number of players vs. time
4. Discuss the results

D.4.2.0.3 Analysis Questions:

- Does doubling the number of players cut the time in half?
- What makes it hard to keep getting faster as you add more players?
- Can you think of tasks where adding more people doesn't help much?

D.4.3 Activity 3: Working Together vs. Working Separately

D.4.3.0.1 Objective: Understand when it's better to work together versus work separately.

D.4.3.0.2 Instructions:

1. Play multiplayer mode with different strategies:
   - **Strategy A**: Each player sorts their own cards independently, then combine at the end
   - **Strategy B**: Players frequently pass cards to each other to help balance the work
2. Record your time and count how many times cards were passed between players
3. Compare which strategy was faster

D.4.3.0.3 Discussion:

- Which strategy was faster?
- Does passing cards between players take time? How much?
- When is it worth the time to pass cards to another player?
- Can you think of real-world situations where coordination takes more time than it saves?

D.5 Exercises

These exercises are designed for students aged 12-16 with no prior programming experience. They focus on understanding concepts through the game rather than technical implementation.

D.5.1 Exercise 1: Comparing Single and Multiple Players

D.5.1.0.1 Task: Play the game twice with the same number of cards (e.g., 30 cards):

- First time: Single-player mode
- Second time: Multiplayer mode with 2-3 friends

D.5.1.0.2 Questions:

1. Which mode finished faster?
2. Was multiplayer always faster? If not, why not?
3. What made working together harder than working alone?
4. If you added more players, would it keep getting faster forever? Why or why not?

D.5.2 Exercise 2: Finding the Best Team Size

D.5.2.0.1 Task: Try sorting 50 cards with different numbers of players and record your time:

- 1 player (you alone)
- 2 players
- 3 players
- 4 players

D.5.2.0.2 Questions:

1. Which team size was fastest?
2. Did more players always mean faster sorting?
3. What problems happened when you had too many players?
4. Can you think of real-world situations where having too many helpers makes things slower?

D.5.3 Exercise 3: Dividing the Work

D.5.3.0.1 Task: Play multiplayer mode and try two different strategies:

- **Strategy A**: Each player takes an equal number of cards at the start and sorts them
- **Strategy B**: Players help whoever has the most cards remaining

D.5.3.0.2 Questions:

1. Which strategy felt easier to coordinate?
2. Which strategy finished faster?
3. What happens if one player is much faster than the others?
4. How does this relate to group projects in school?

D.6 Conceptual Mappings

D.6.1 Game to OpenMP

Table D.1: Detailed game-to-OpenMP concept mapping

Game Element	OpenMP Concept	Code Example
Single-player session	Parallel region	`#pragma omp parallel`
Main card container	Shared array	Global array variable
Buffer zones	Thread-private storage	`private(buffer)`
No forced turn-taking	Independent threads	No explicit synchronization
Sorting within buffer	Local computation	Serial code in parallel region
Merging back to main	Reduction/merge	`#pragma omp critical`
Completion check	Barrier synchronization	`#pragma omp barrier`

D.7 Assessment Rubric

For instructors wishing to grade student understanding:

D.7.1 Conceptual Understanding (40 points)

- (10 pts) Can identify shared-memory vs. distributed-memory paradigms
- (10 pts) Understands role of communication in parallel performance
- (10 pts) Can explain speedup and scalability concepts
- (10 pts) Recognizes load balancing importance

D.7.2 Practical Application (30 points)

- (15 pts) Designs reasonable parallel sorting strategy
- (15 pts) Implements parallel sorting code (OpenMP)

D.7.3 Analysis and Reflection (30 points)

- (10 pts) Analyzes performance data from experiments
- (10 pts) Connects game experience to real HPC systems
- (10 pts) Reflects on limitations and trade-offs

D.8 Additional Resources

D.8.1 Recommended Readings

1. Pacheco, P. (2011). *An Introduction to Parallel Programming.* Morgan Kaufmann.
2. OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface.*
3. Chapman, B., Jost, G., & Van Der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming.* MIT Press.

D.8.2 Online Resources

- OpenMP Tutorials: `https://www.openmp.org/resources/tutorials-articles/`
- Parallel Computing Course Materials: `https://www.cs.cmu.edu/ 15418/`
- Lawrence Livermore National Lab OpenMP Tutorial: `https://hpc.llnl.gov/tuts/openMP/`

D.8.3 Hands-On Practice

- Try the game with different configurations
- Implement actual parallel sorting code
- Profile and optimize parallel programs
- Experiment with HPC clusters (if available)

D.9 Instructor Notes

D.9.1 Integration with Curriculum

D.9.1.0.1 As Pre-Lecture Activity (15–20 minutes):

- Students play game before formal HPC lecture
- Generates curiosity and questions
- Provides concrete reference points for abstract concepts
- Follow with discussion connecting gameplay to course material

D.9.1.0.2 As Lab Exercise (50–90 minutes):

- Structured activities with data collection
- Analysis of performance metrics
- Written reflection connecting to theory
- Implementation of actual parallel code based on strategies

D.9.1.0.3 As Homework/Assessment:

- Students play independently, record observations
- Answer reflection questions
- Design parallel algorithms inspired by gameplay
- Optional: Implement and benchmark solutions

D.9.2 Common Student Misconceptions

Be prepared to address:

1. **"More processes/threads always means faster"**: Discuss communication overhead and Amdahl's Law
2. **"Shared memory means no synchronization needed"**: Explain race conditions and critical sections
3. **"Message passing is always slower"**: Discuss scalability benefits of distributed memory
4. **"The game perfectly simulates HPC"**: Clarify simplifications and abstractions